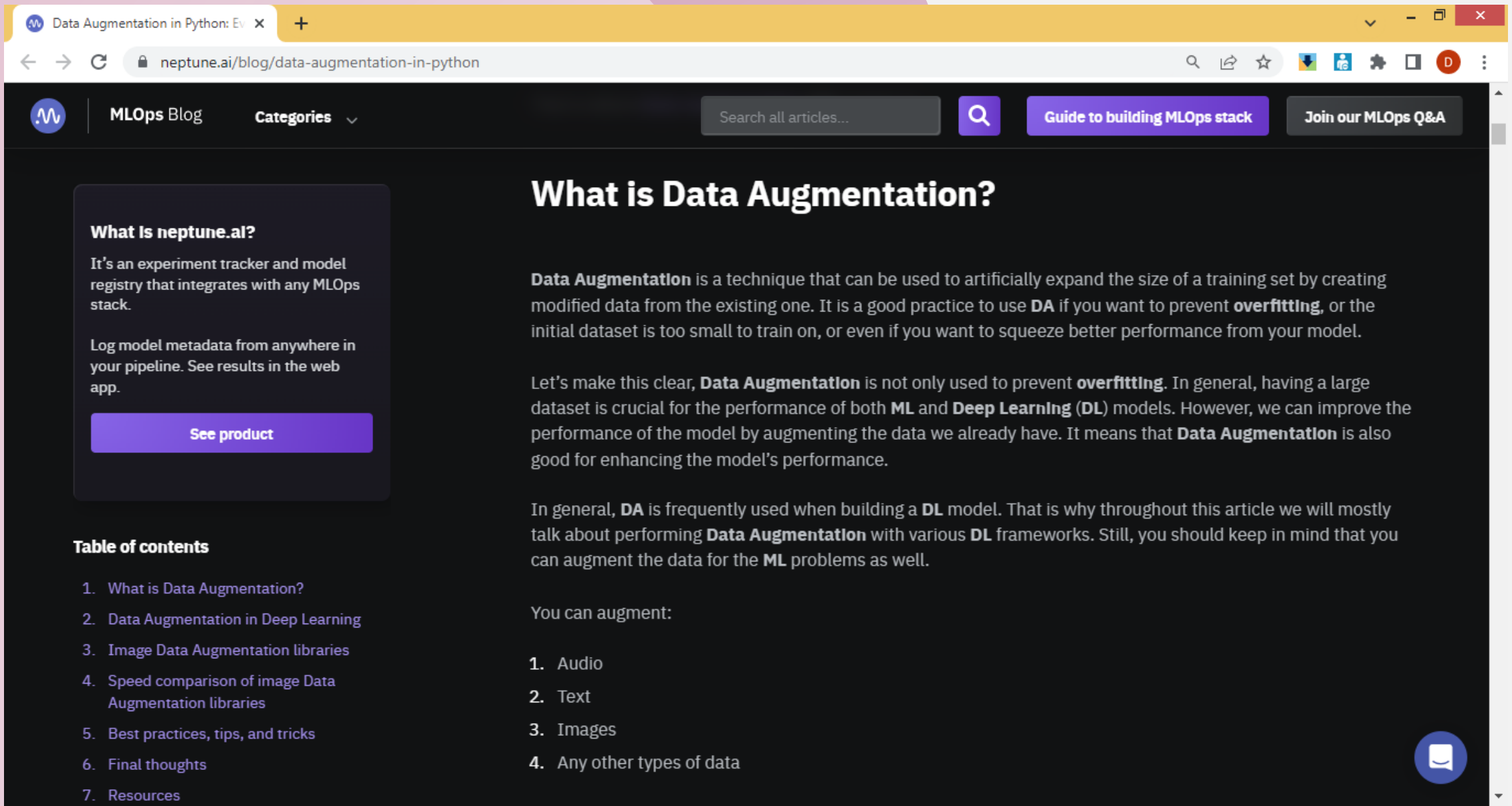
TĂNG CƯỜNG DỮ LIỆU

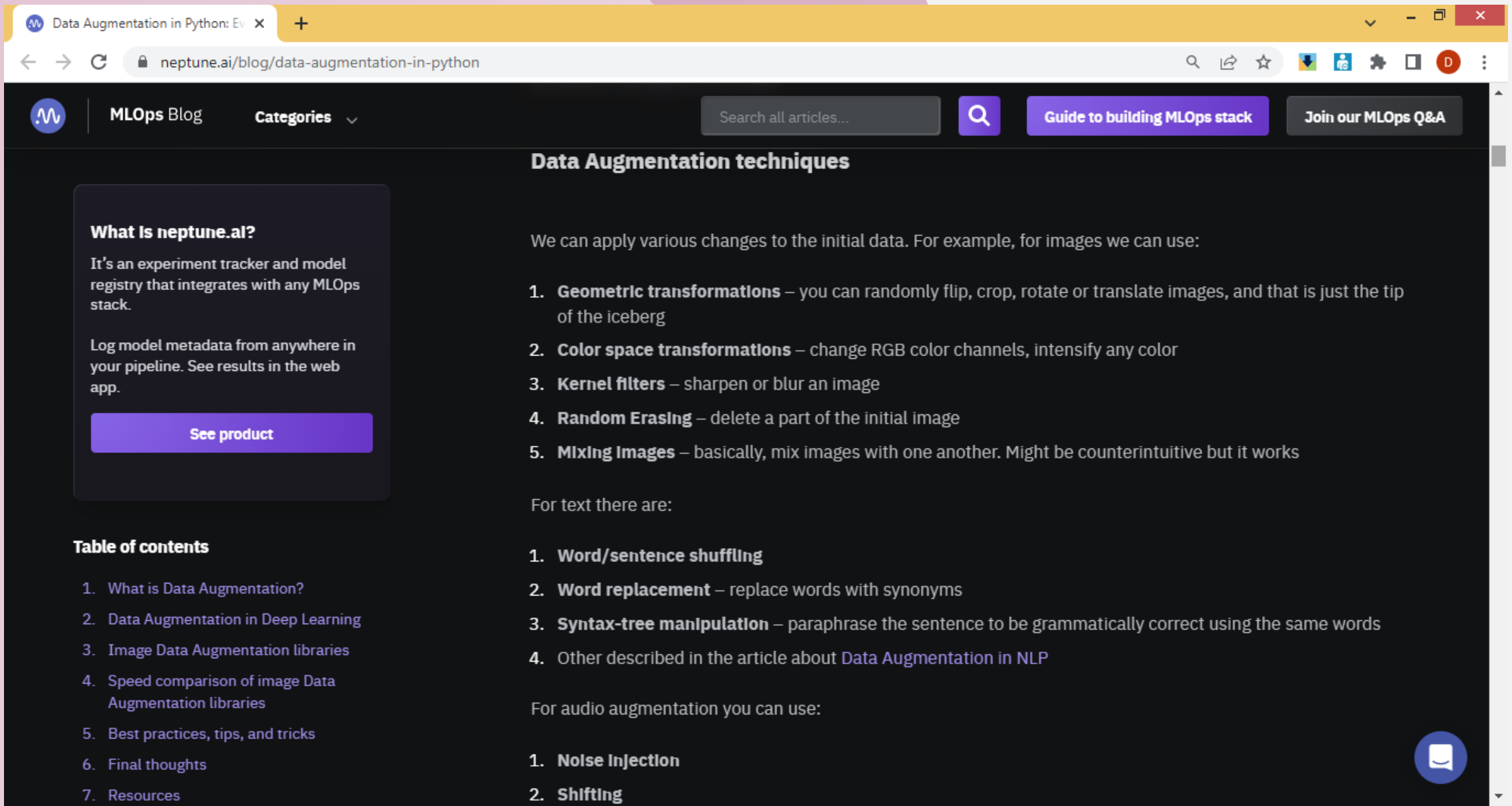
DATA AUGMENTATION



https://neptune.ai/blog/data-augmentation-in-python



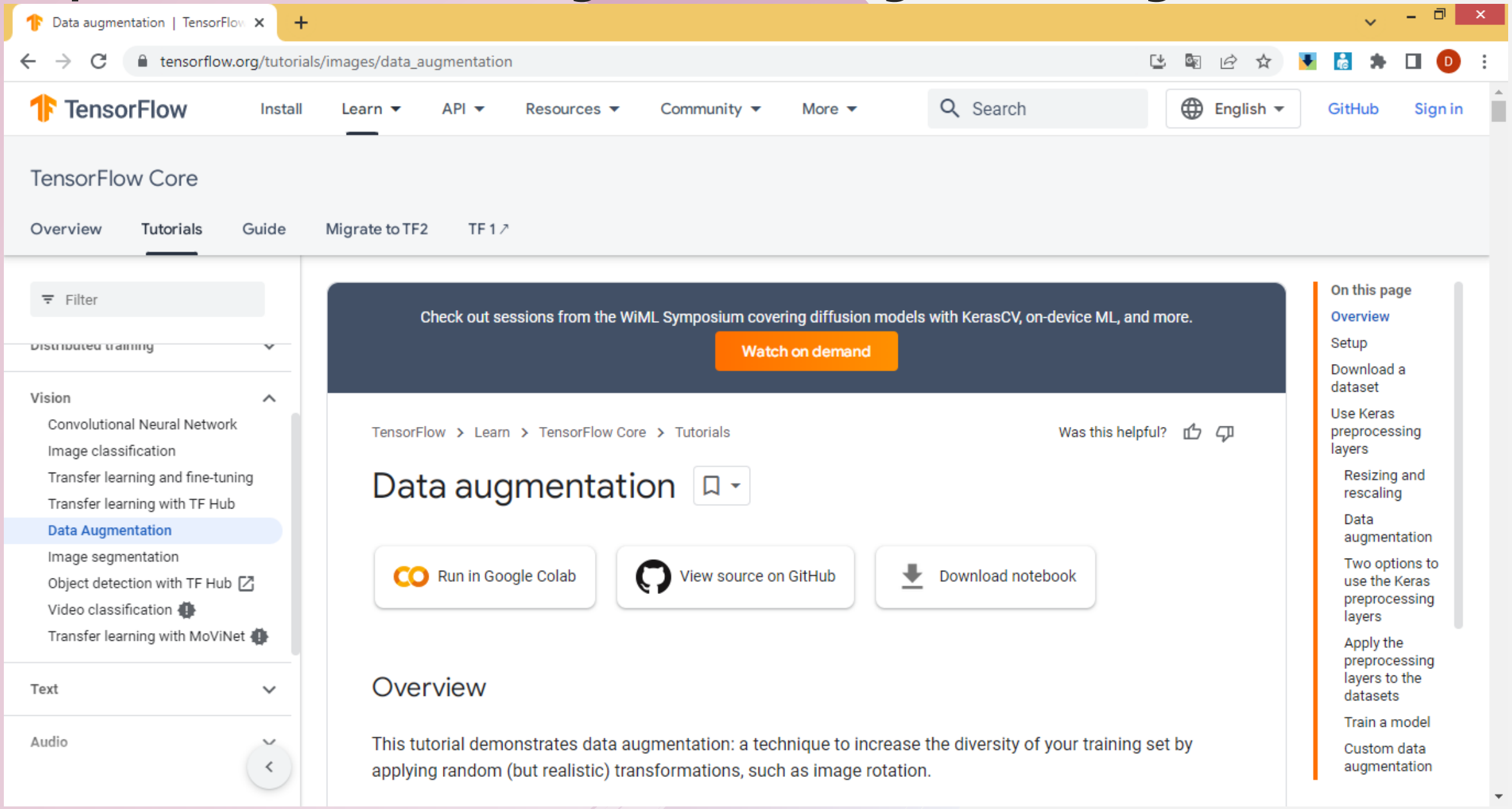
https://neptune.ai/blog/data-augmentation-in-python



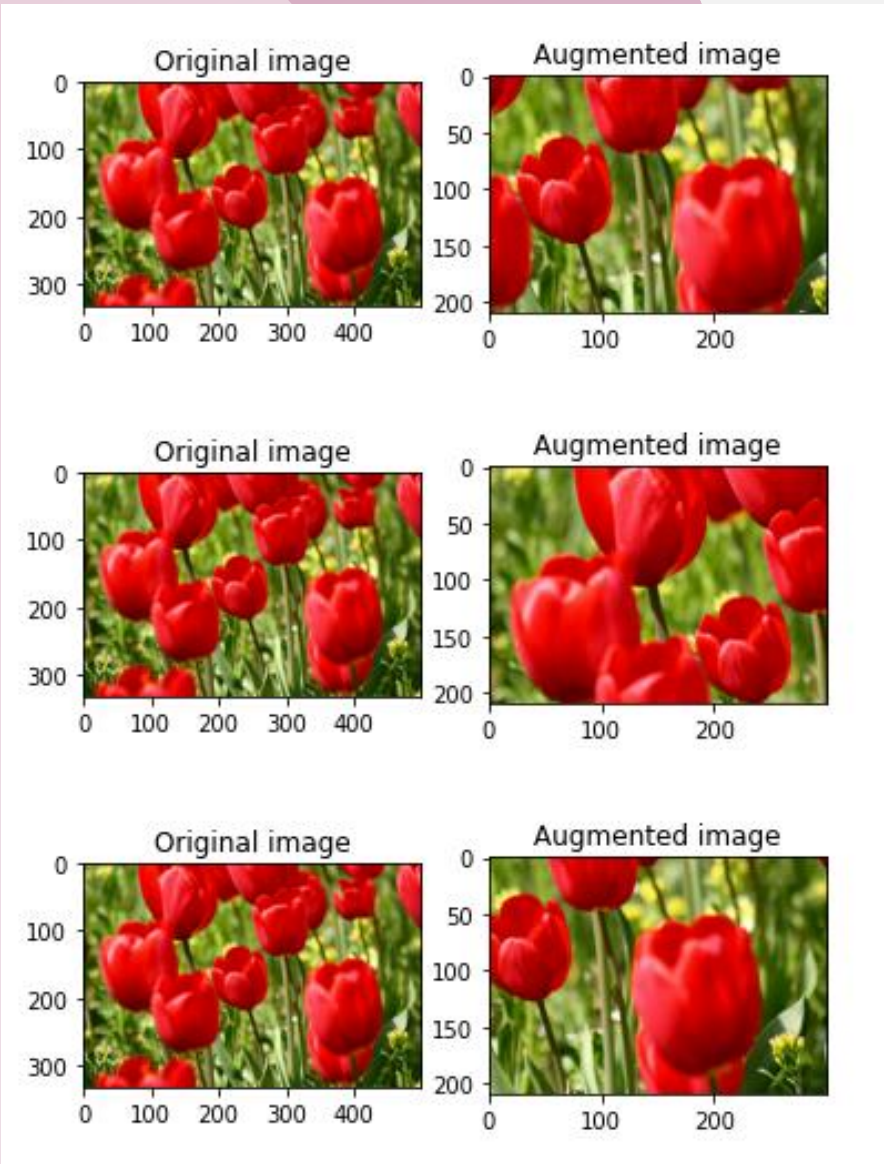
https://neptune.ai/blog/data-augmentation-in-python



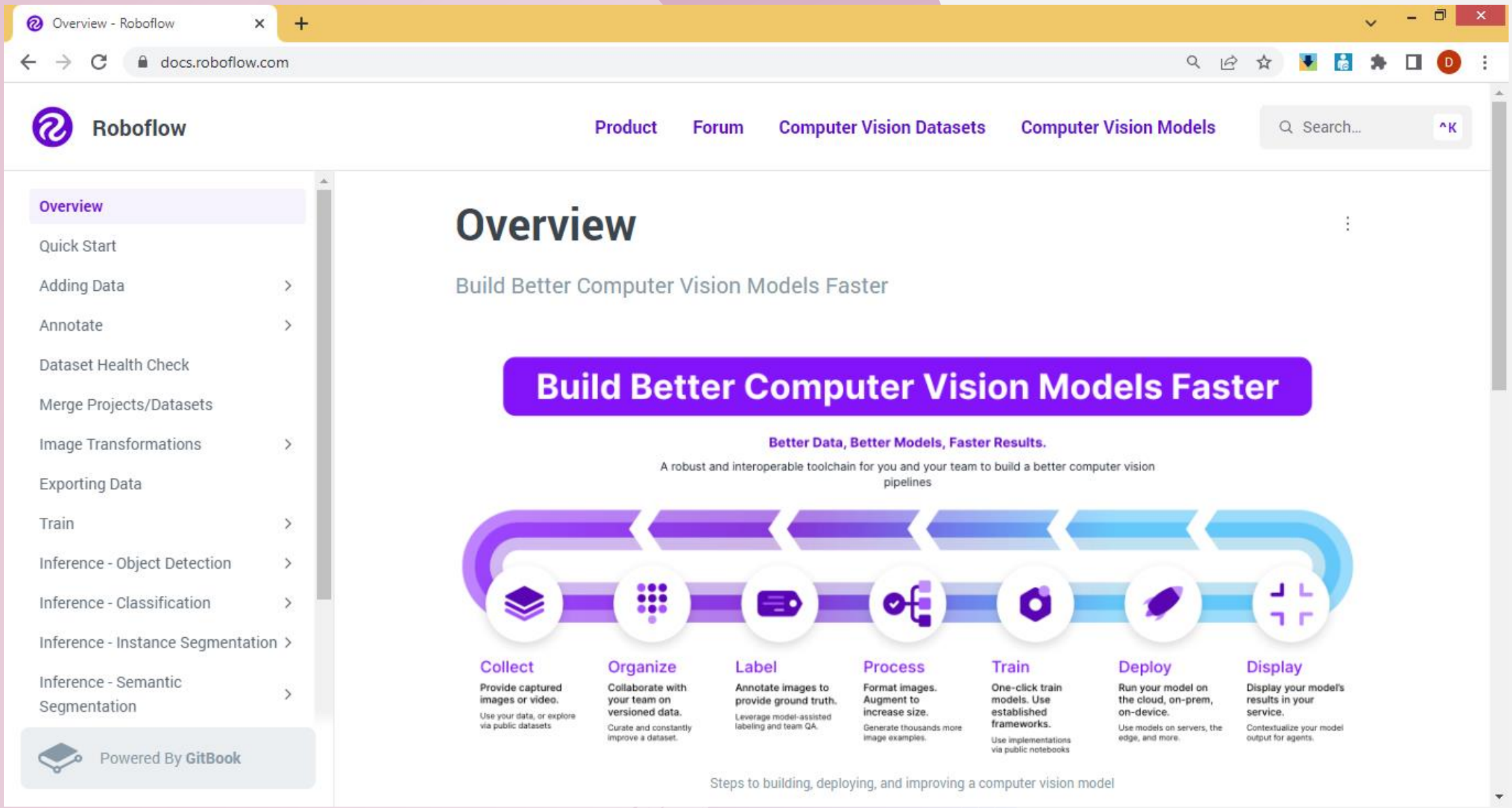
https://www.tensorflow.org/tutorials/images/data_augmentation

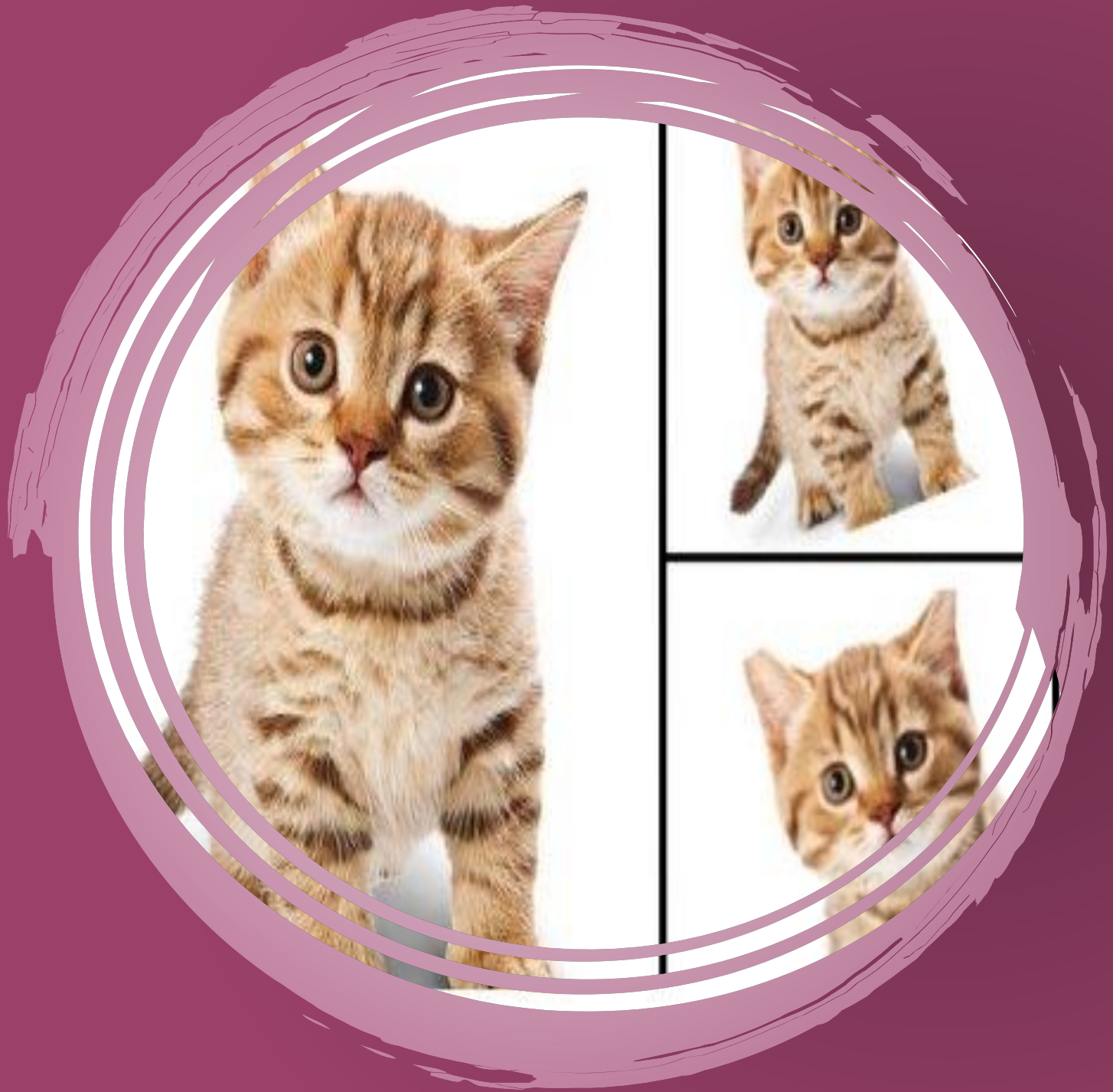


https://www.tensorflow.org/tutorials/images/data_augmentation



https://docs.roboflow.com/



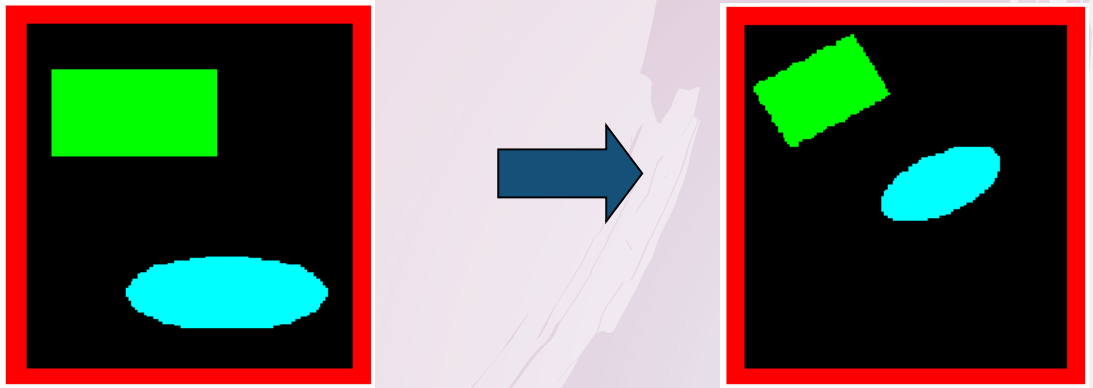


CÁC PHÉP BIẾN ĐỔI HÌNH HỌC

IMAGE GEOMETRY

Geometric Operations

- **Scale** - change image content size
- **Rotate** - change image content orientation
- **Reflect** - flip over image contents
- **Translate** - change image content position
- **Affine Transformation**
  - general image content linear geometric transformation



Geometric Operations





http://what-when-how.com/introduction-to-video-and-image-processing/geometric-transformations-introduction-to-video-and-image-processing-part-1/

Geometric transformations

- Geometric transformations are common in computer graphics, and are often used in image analysis.
- Geometric transforms permit the elimination of geometric distortion that occurs when an image is captured.
- If one attempts to match two different images of the same object, a geometric transformation may be needed.
- Applying data augmentation is to increase the generalizability of the model.
- Examples?

tf.keras.preprocessing.image.ImageDataGenerator

TensorFlow 1 version

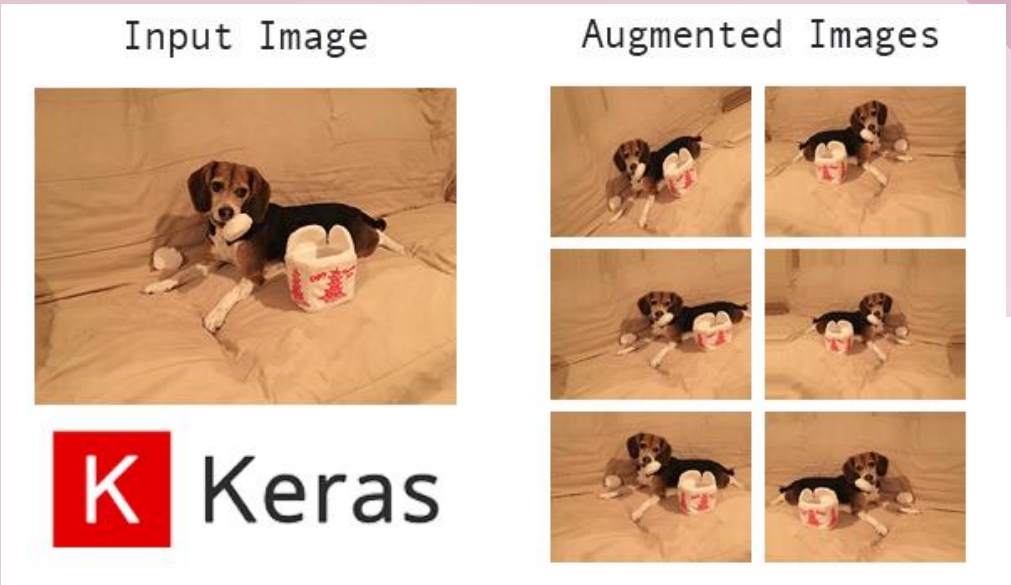
View source on GitHub

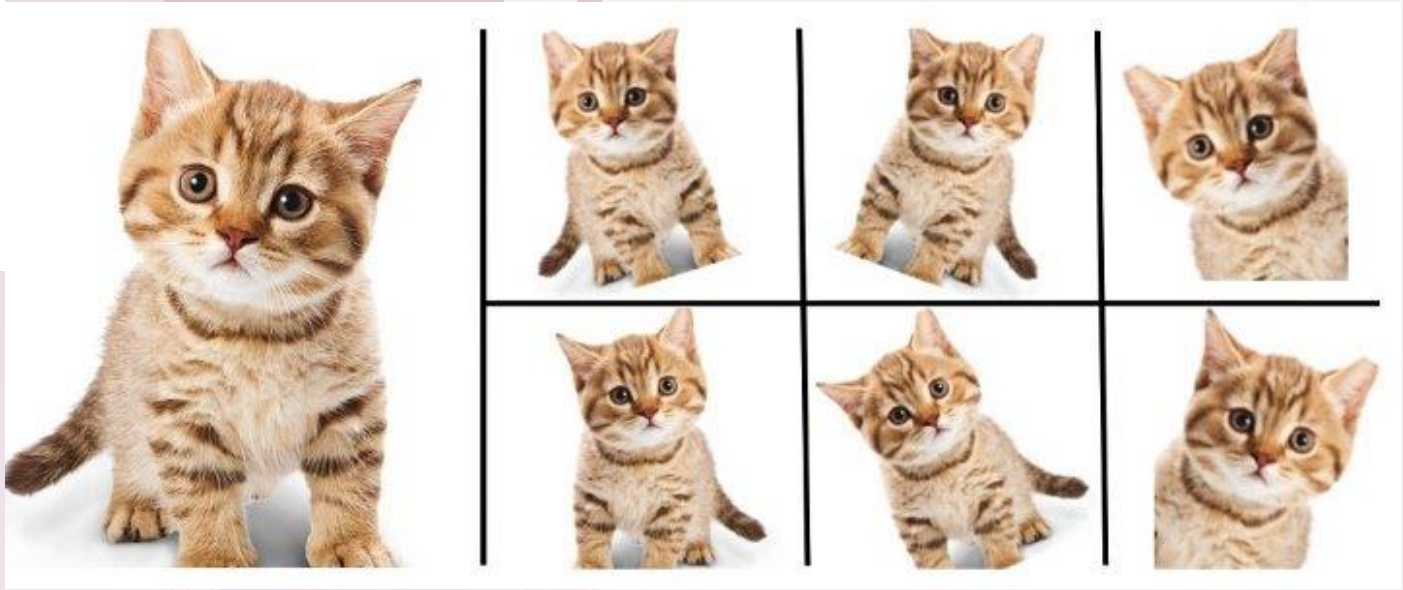
Generate batches of tensor image data with real-time data augmentation.

View aliases

```
tf.keras.preprocessing.image.ImageDataGenerator(
    featurewise_center=False, samplewise_center=False,
    featurewise_std_normalization=False, samplewise_std_normalization=False,
    zca_whitening=False, zca_epsilon=1e-06, rotation_range=0, width_shift_range=0.0,
    height_shift_range=0.0, brightness_range=None, shear_range=0.0, zoom_range=0.0,
    channel_shift_range=0.0, fill_mode='nearest', cval=0.0,
    horizontal_flip=False, vertical_flip=False, rescale=None,
    preprocessing_function=None, data_format=None, validation_split=0.0, dtype=None
)
```

https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator

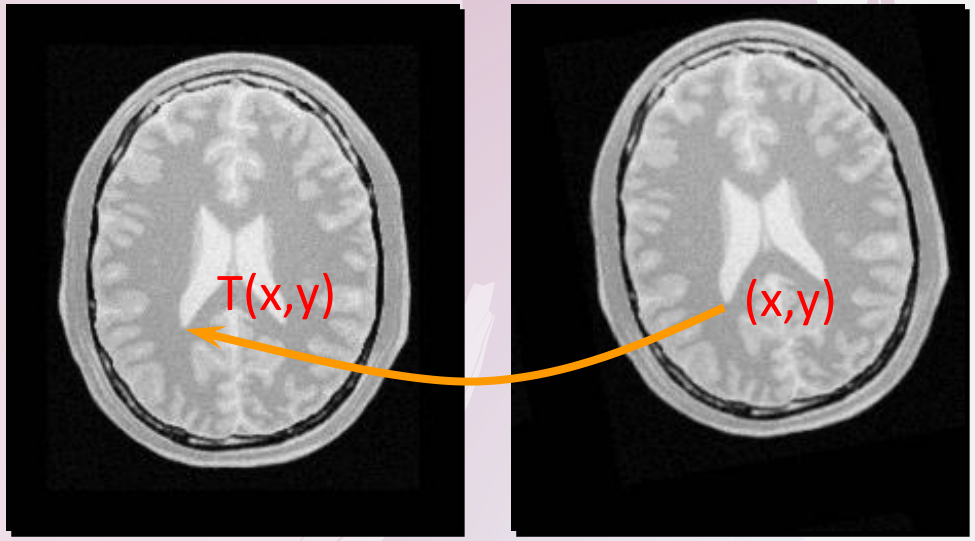




https://www.pyimagesearch.com/2019/07/08/keras-imagedatagenerator-and-data-augmentation/
https://machinelearningknowledge.ai/keras-imagedatagenerator-for-image-augmentation/

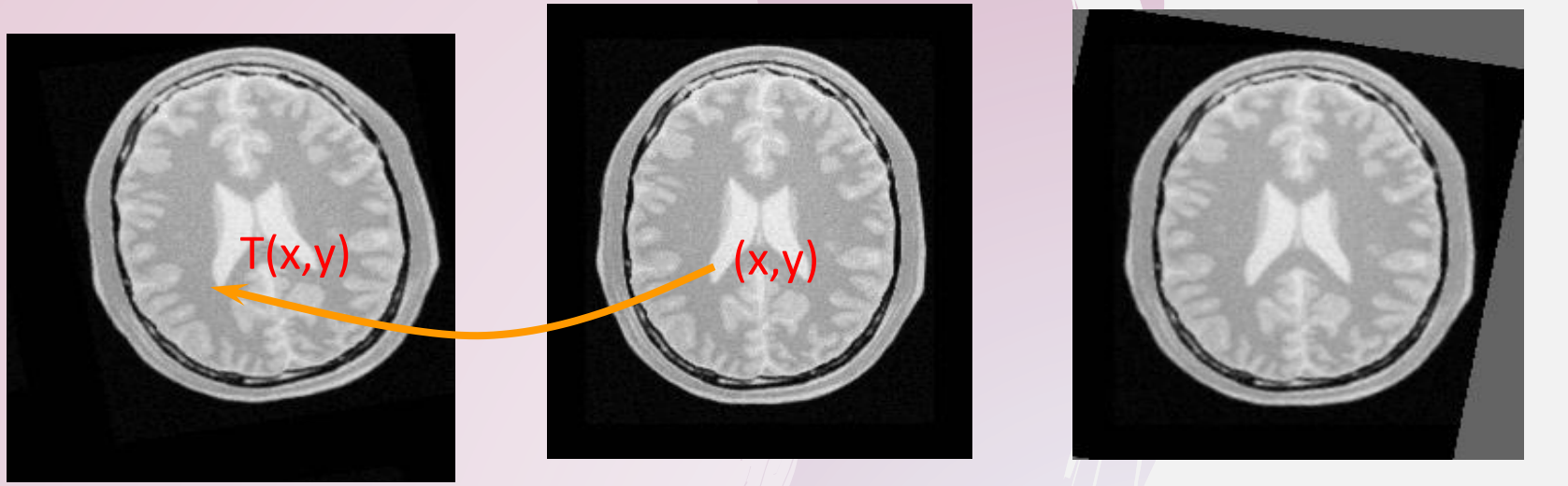
Geometric Transformations

- A geometric transform consists of two basic steps ...
  - Step1: determining the pixel co-ordinate transformation
    - mapping of the co-ordinates of the moving image pixel to the point in the fixed image.



Geometric transformations

- Step2: determining the brightness of the points in the digital grid of the transformed image.
  - brightness is usually computed as an interpolation of the brightnesses of several points in the neighborhood.

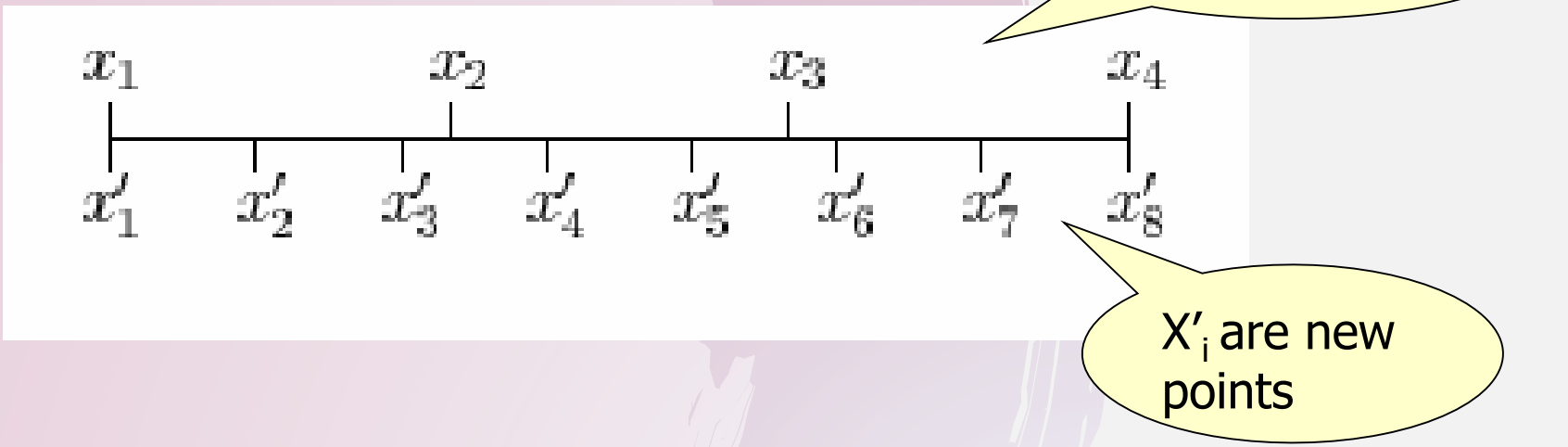


Fixed Image Moving Image xformed Moving Image

We'll discuss step 2 first.

Interpolation of Data

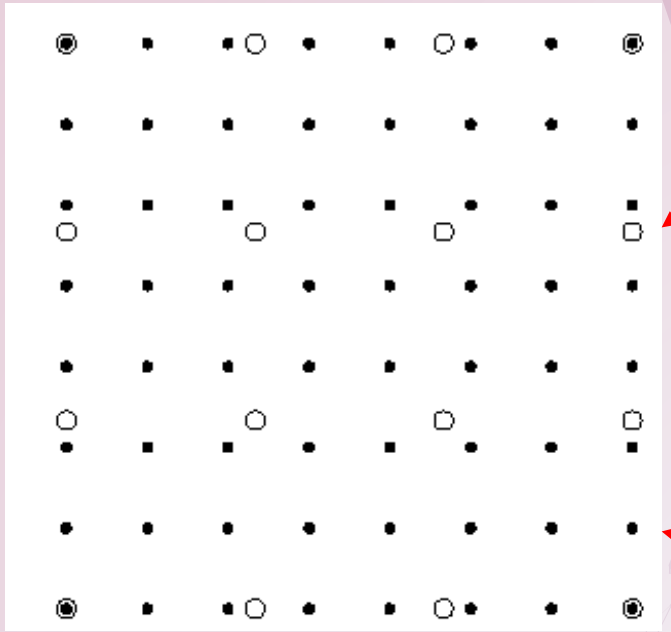
- Given a function $f(x)$ at 4 points, how to "guess" values at other points?



- Guessing at the function values within the known range is called interpolation.
- Interpolation has great significance in general image/video processing.

Another Example

Interpolation on an image (4x4 -> 8x8) after scaling

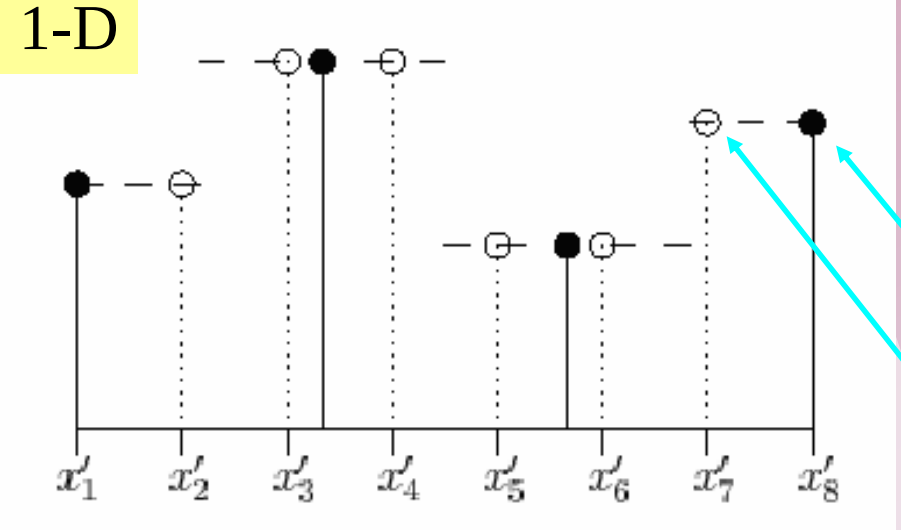


Open circle:
Original image pixel

Closed circle:
New pixels

Interpolation: Nearest Neighbor

1-D



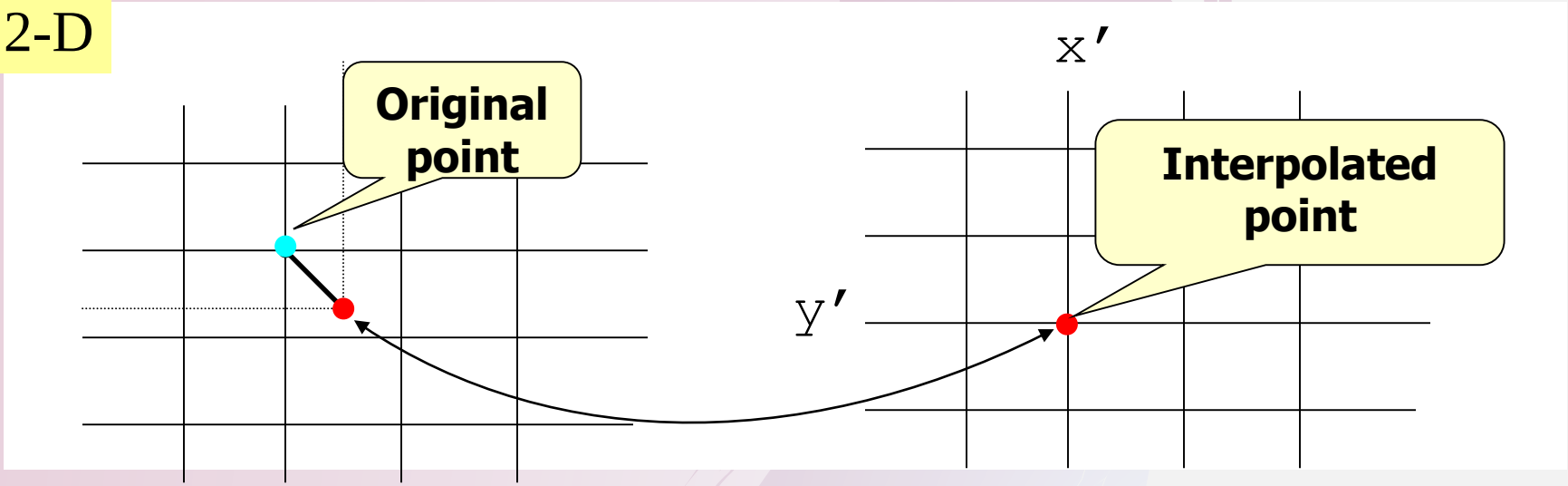
We assign $f(x_i') = f(x_j)$

x_j is the original point closest to x_i'

The original function values

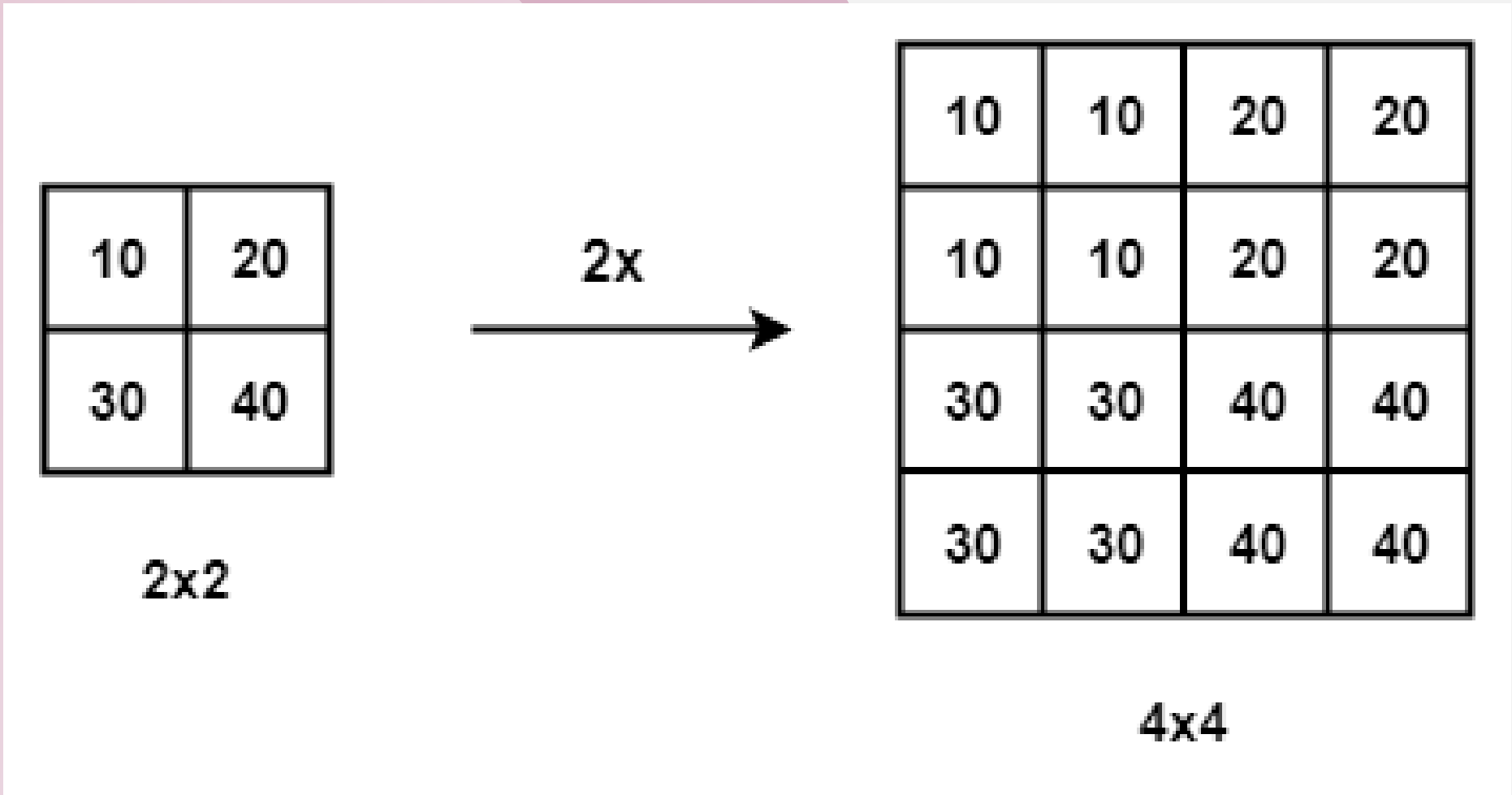
The interpolated values

2-D



setting the pixel value on interpolated point to the pixel of closet image point

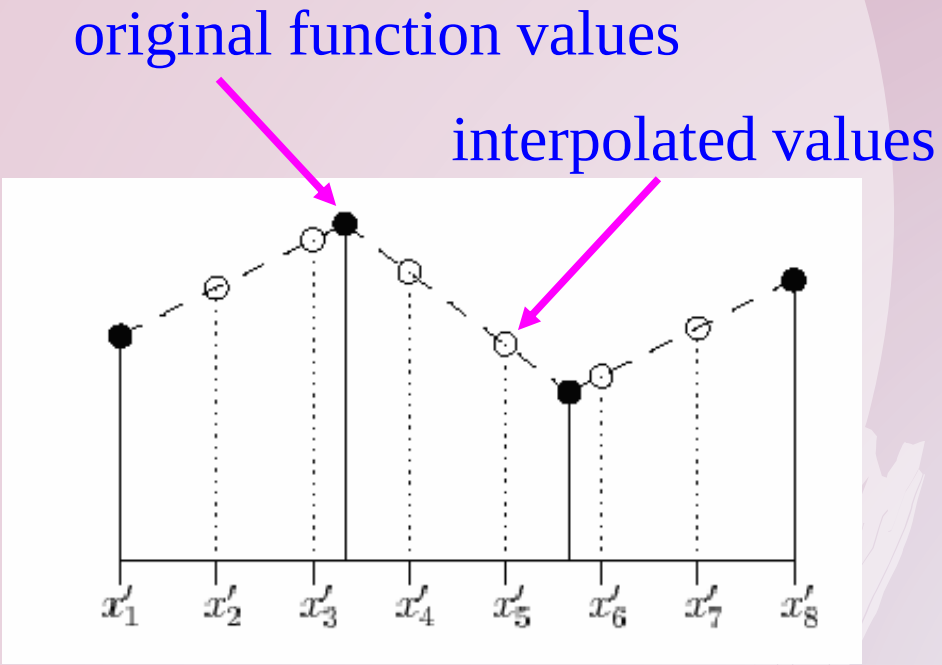
Interpolation: Nearest Neighbor



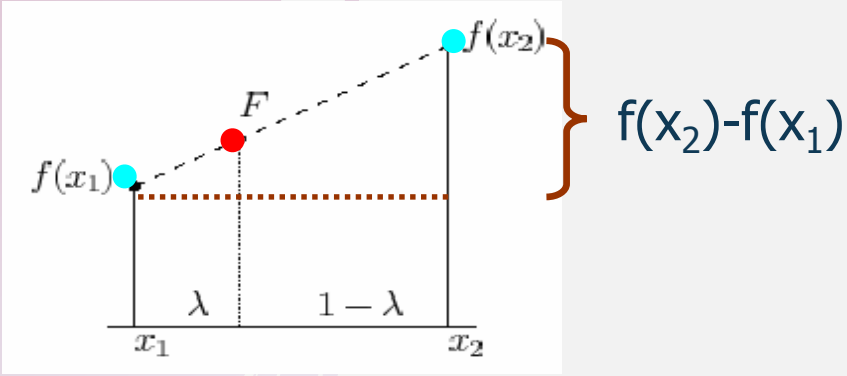
setting the pixel value on interpolated point to the pixel of closet image point

Interpolation: Linear (1D)

- General idea:



To calculate the interpolated values

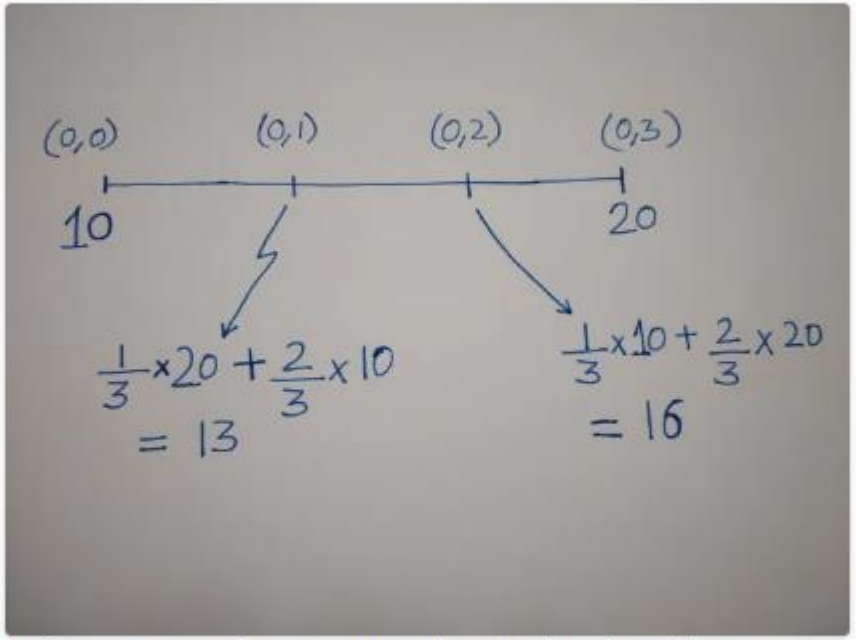


$$\frac{F - f(x_1)}{\lambda} = \frac{f(x_2) - f(x_1)}{1}$$

Interpolation: Linear (1D)

- General idea:

Linear interpolation means we estimate the value using linear polynomials. Suppose we have 2 points having value 10 and 20 and we want to guess the values in between them. Simple Linear interpolation looks like this

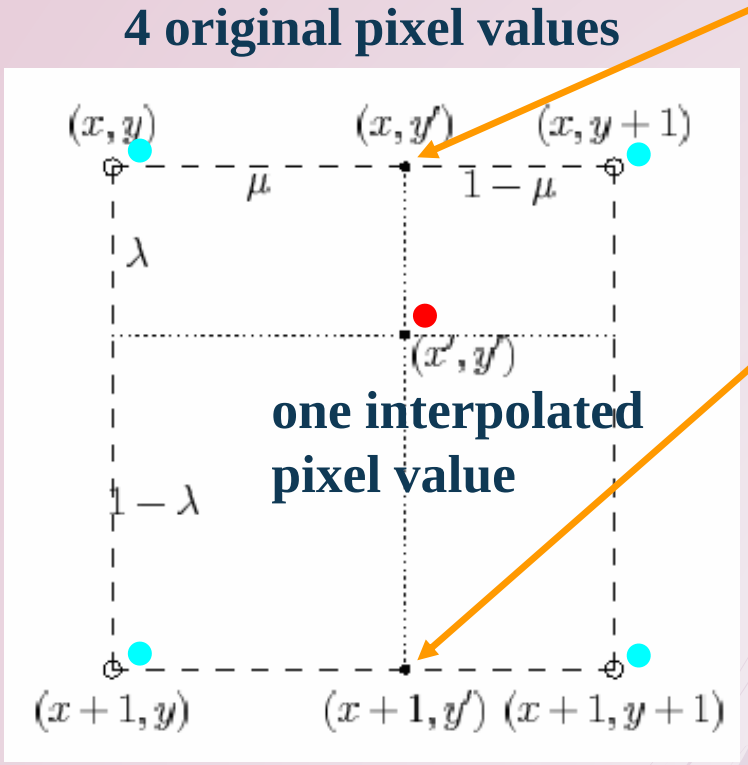


More weight is given to the nearest value(See 1/3 and 2/3 in the above figure). For 2D (e.g. images), we have to perform this operation twice once along rows and then along columns that is why it is known as Bi-Linear interpolation.

https://theailearner.com/2018/12/29/image-processing-bilinear-interpolation/

Interpolation: Linear (2D)

- How a 4x4 image would be interpolated to produce an 8x8 image?



$$f(x, y') = \mu f(x, y+1) + (1-\mu) f(x, y)$$

$$f(x+1, y') = \mu f(x+1, y+1) + (1-\mu) f(x+1, y)$$

Along the y' column we have

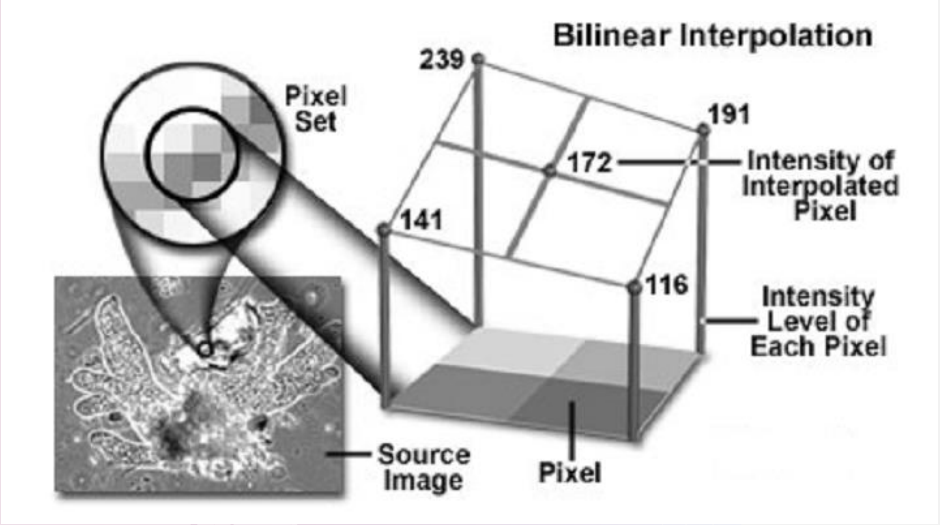
$$f(x', y') = \lambda f(x+1, y') + (1-\lambda) f(x, y')$$

Bilinear Interpolation

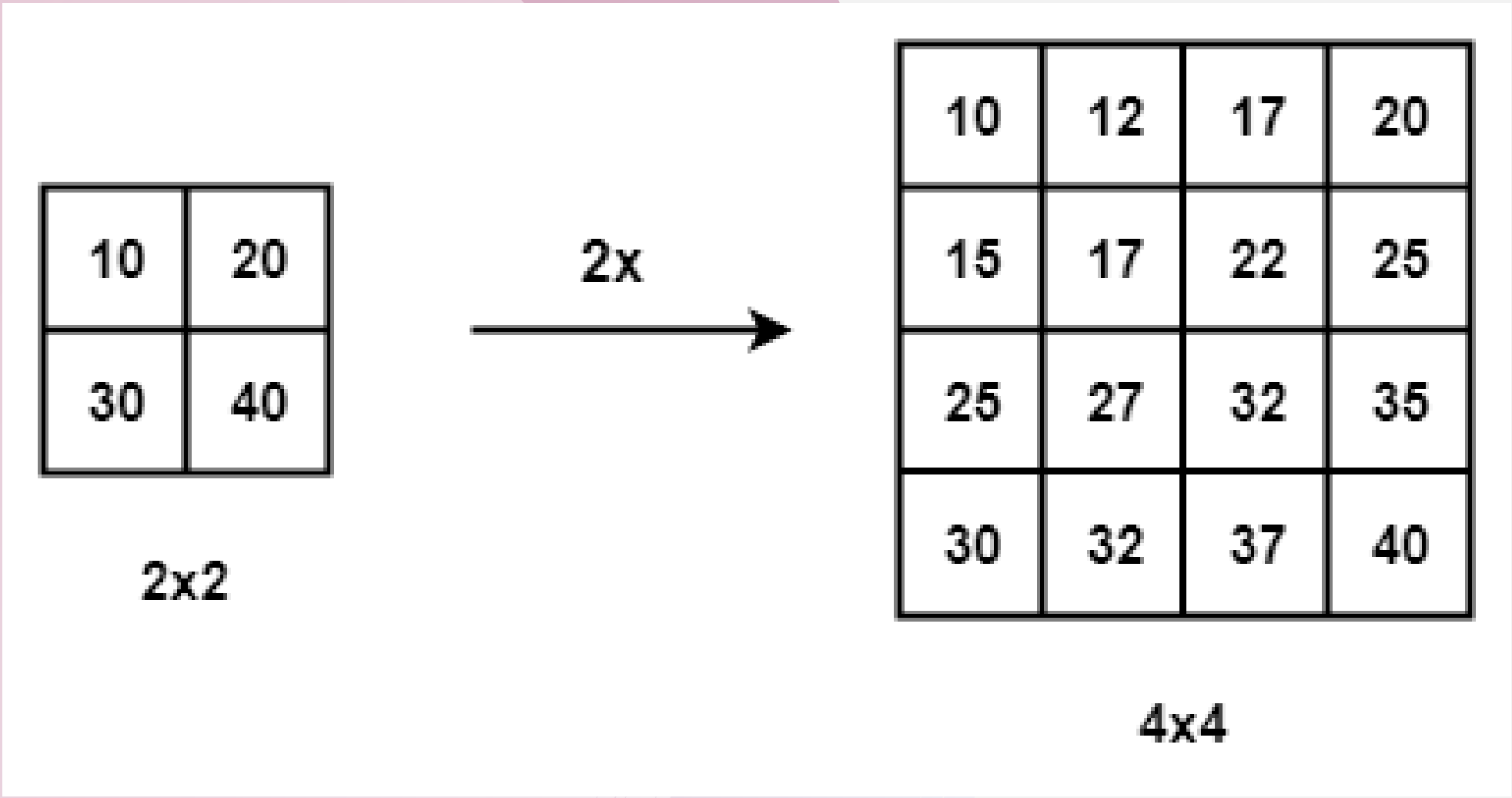
- Substituting with the values just obtained:

$$f(x',y') = \lambda(\mu f(x+1,y+1) + (1-\mu) f(x+1,y)) + (1-\lambda)(\mu f(x,y+1) + (1-\mu) f(x,y))$$

- You can do the expansion as an exercise.
- This is the formulation for bilinear interpolation

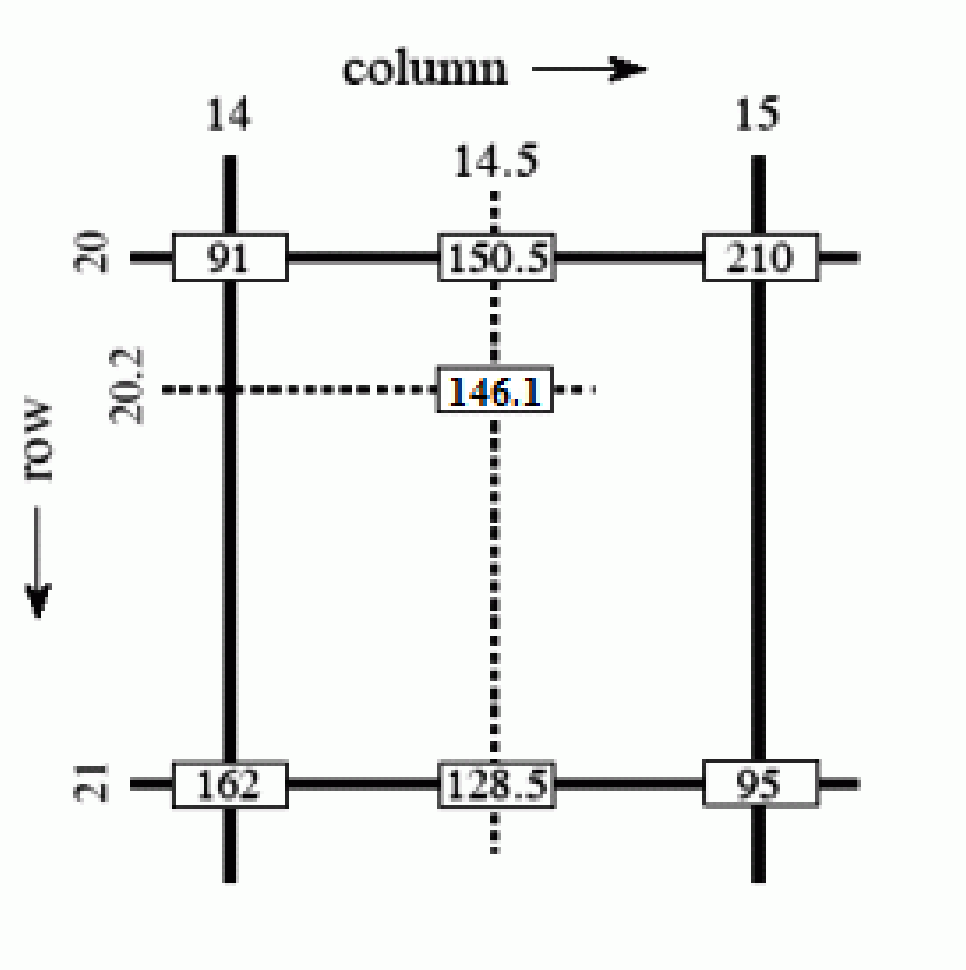


Bilinear Interpolation



Bilinear Interpolation

- Example of bilinear interpolation

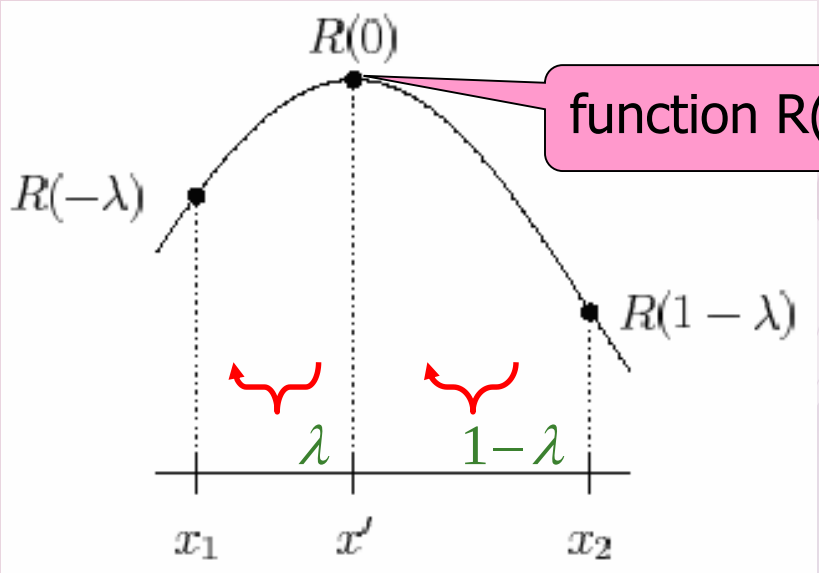


https://upload.wikimedia.org/wikipedia/commons/a/a7/Bilin3.png

General Interpolation

- We wish to interpolate a value f(x') for and suppose $x_1 \le x' \le x_2$
- We define an interpolated value R(u) and set $x' - x_1 = \lambda$ ← $0 \le \lambda \le 1$

$$f(x') = R(-\lambda) f(x_1) + R(1-\lambda) f(x_2)$$

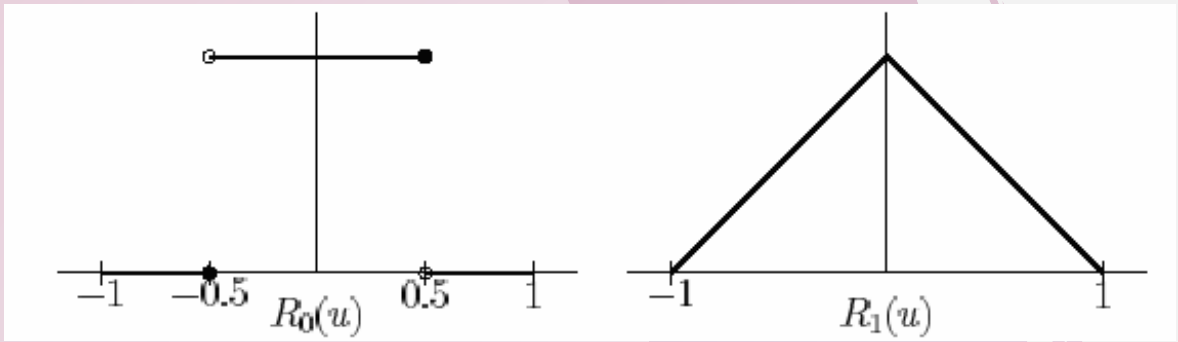


function R(u) is centered at x'

x_1 corresponds with u= - λ,
and x_2 with u= 1- λ

General Interloplation: 0th and 1st orders

- Consider 2 functions $R_0(u)$ and $R_1(u)$



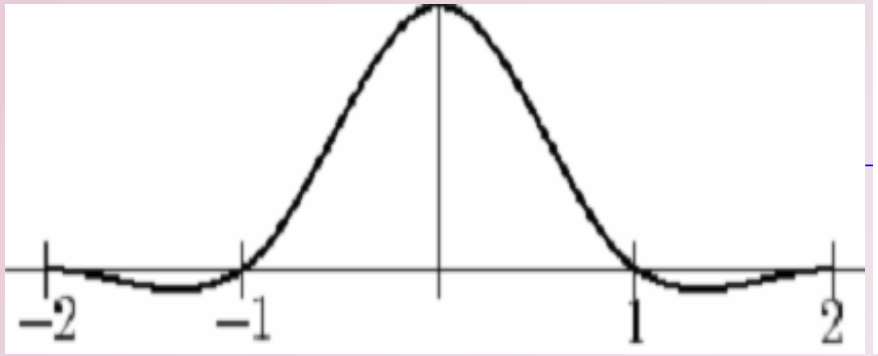
$$R_0(u) = \begin{cases} 0 & if\ u \le -0.5 \\ 1 & if\ -0.5 < u \le 0.5 \\ 0 & if\ u > 0.5 \end{cases} \qquad R_1(u) = \begin{cases} 1+u & if\ u \le 0 \\ 1-u & if\ u \ge 0 \end{cases}$$

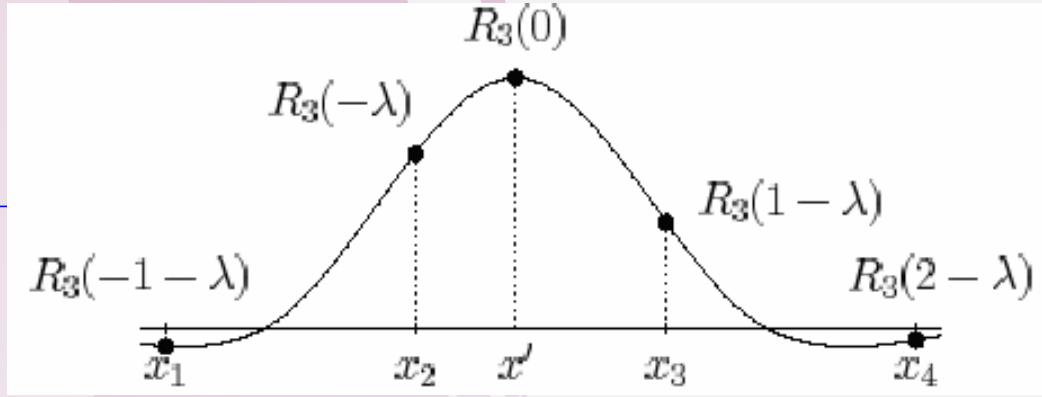
Substitute $R_0(u)$ for R(u) ⇒ nearest neighbour interpolation.

Substitute $R_1(u)$ for R(u) ⇒ linear interpolation.

General Interloplation: 3rd order (Cubic)

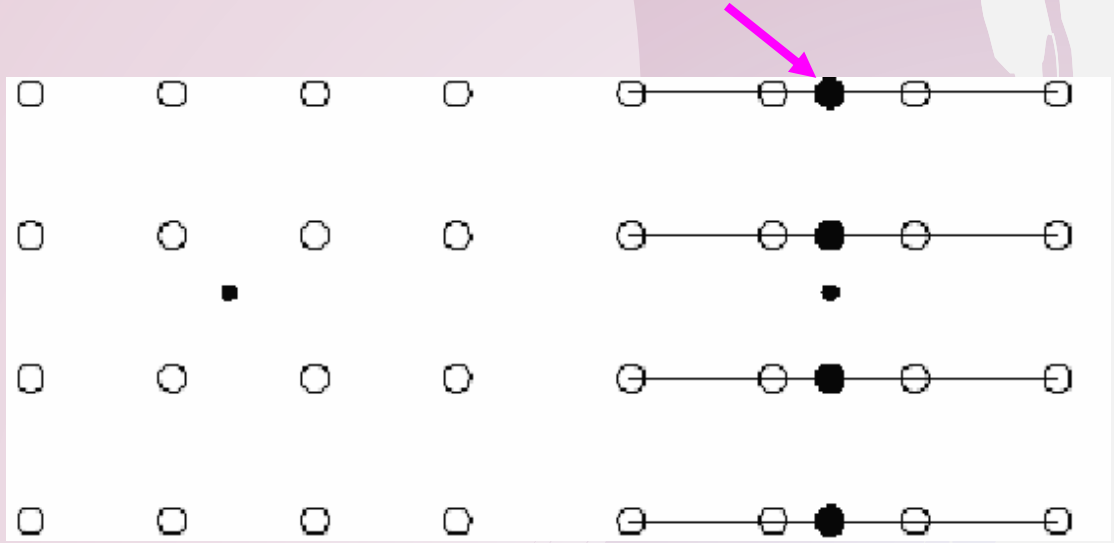
$$R_3(u) = \begin{cases} 1.5|u|^3 - 2.5|u|^2 + 1 & if\ |u| \le 1 \\ -0.5|u|^3 + 2.5|u|^2 - 4|u| + 2 & if\ 1 < |u| \le 2 \end{cases}$$





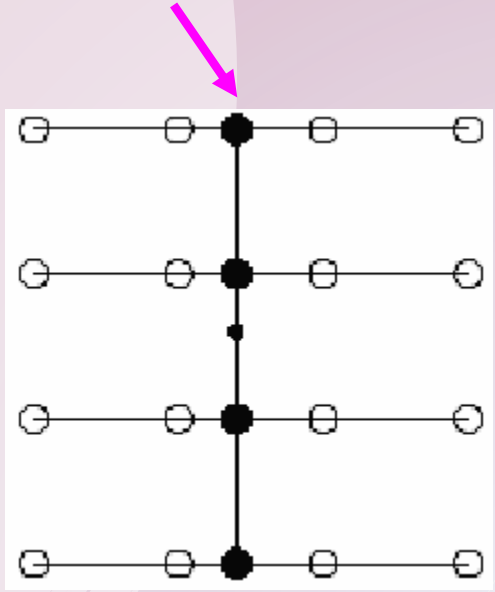
General Interpolation: Bicubic (2D)

- Bicubic interpolation fits a series of cubic polynomials to the brightness values contained in the 4 x 4 array of pixels surrounding the calculated address.
  - Step 1: four cubic polynomials F(i), i = 0, 1, 2, 3 are fit to the control points along the rows. The fractional part of the calculated pixel's address in the x-direction is used.



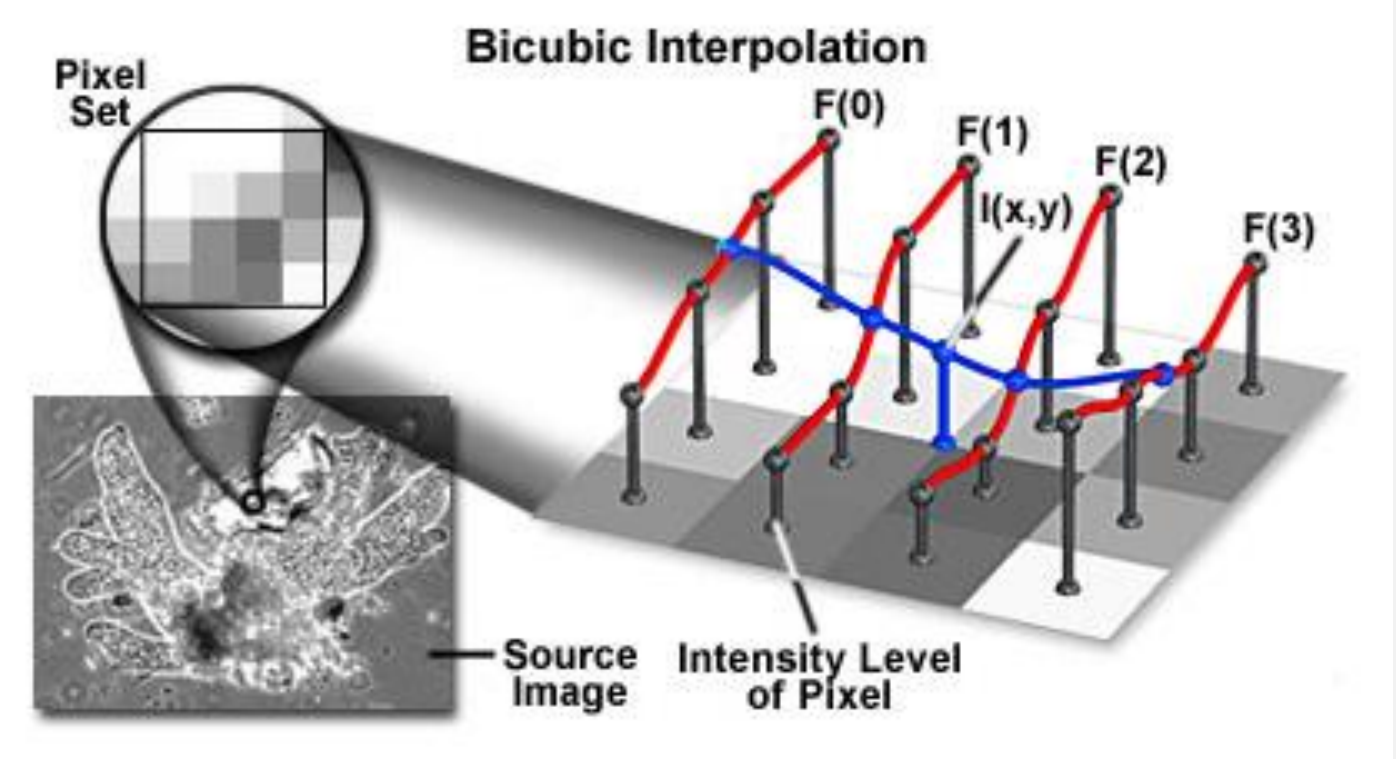
General Interpolation: Bicubic

- Step 2: the fractional part of the calculated pixel's address in the y-direction is used to fit another cubic polynomial down the column, based on the interpolated brightness values that lie on the curves F(i), i = 0, ..., 3.



General Interpolation: Bicubic

- Substituting the fractional part of the calculated pixel's address in the x-direction into the resulting cubic polynomial then yields the interpolated pixel's brightness value.



General Interpolation: Example



- Original detailed part of flower image (8bit,75×75)
- Detailed part of super-resolution image (8bit,300×300) :



NN Interpolation



Bilinear Interpolation



Bicubic Interpolation

```
im = imread('flower.jpg');
im2= imresize(im,[800,800],method);
```

Example

```
1 import cv2
2 import numpy as np
3
4 img = cv2.imread('C:/New folder/apple.jpg')
```

This is a 20×22 apple image that looks like this.



https://theailearner.com/2018/11/15/image-interpolation-using-opencv-python/

Example

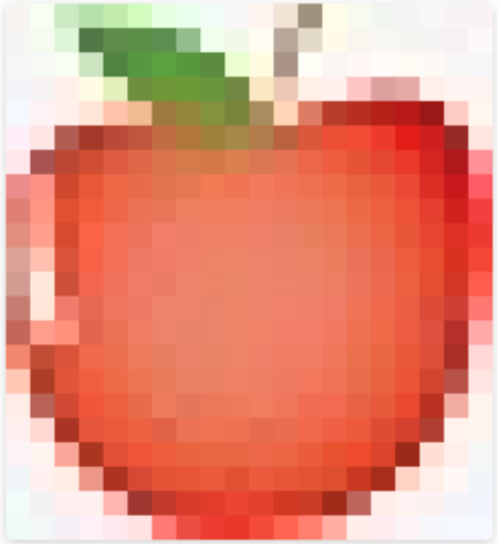


Nearest Neighbor Interpolation

In this we use **cv2.INTER_NEAREST** as the interpolation flag in the cv2.resize() function as shown below

```
1 near_img = cv2.resize(img,None, fx = 10, fy = 10, interpolation = cv2.INTER_NEAREST)
```

Output:



Clearly, this produces a pixelated or blocky image. Also, it doesn't introduce any new data.

https://theailearner.com/2018/11/15/image-interpolation-using-opencv-python/

Example



Bilinear Interpolation

In this we use **cv2.INTER_LINEAR** flag as shown below

```
1 bilinear_img = cv2.resize(img,None, fx = 10, fy = 10, interpolation = cv2.INTER_LINEAR)
```

Output:



This produces a smooth image than the nearest neighbor but the results for sharp transitions like edges are not ideal because the results are a weighted average of 2 surrounding pixels.

https://theailearne

Example



Bicubic Interpolation

In this we use **cv2.INTER_CUBIC** flag as shown below

```
1 bicubic_img = cv2.resize(img,None, fx = 10, fy = 10, interpolation = cv2.INTER_CUBIC)
```

Output:



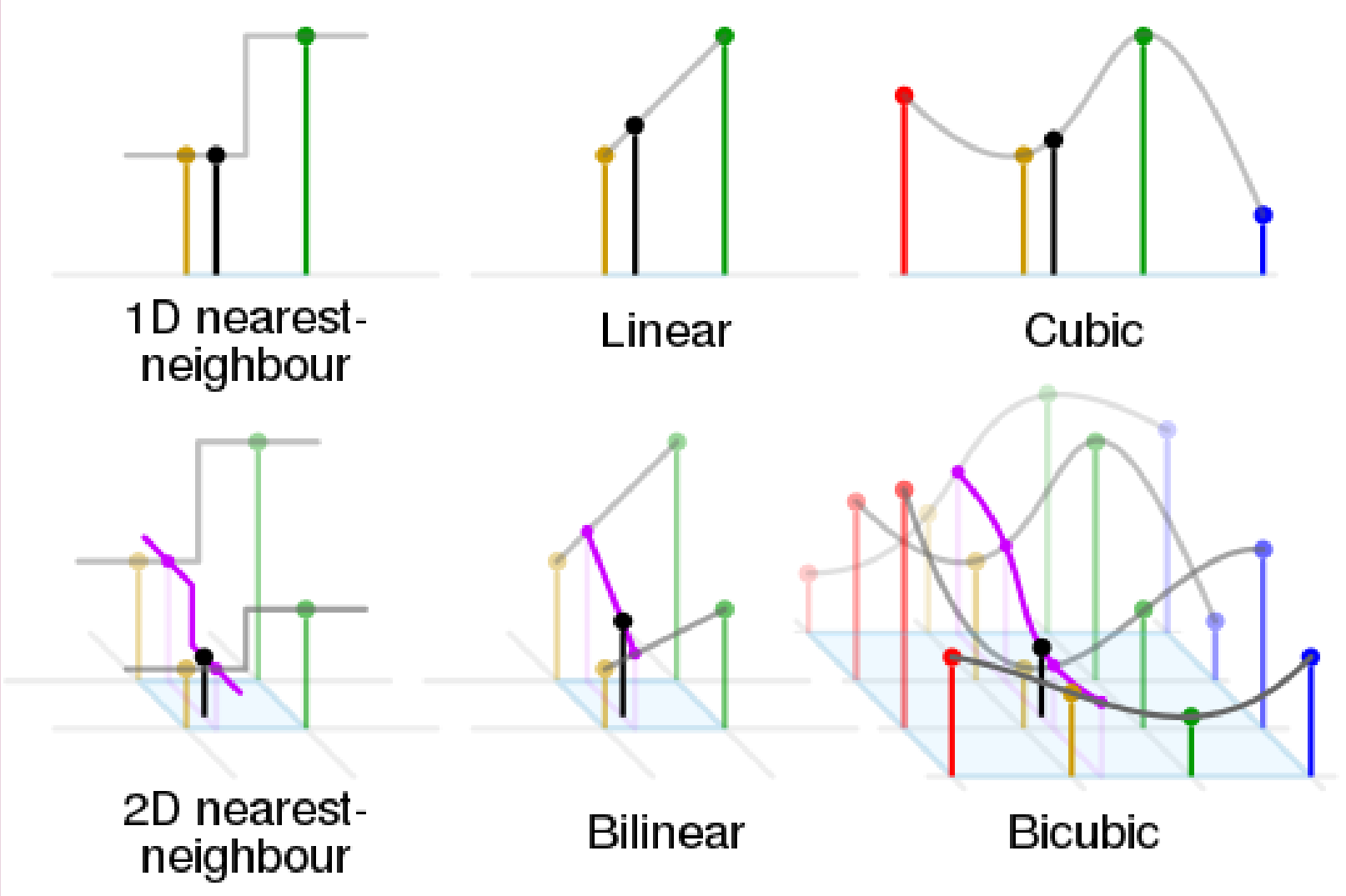
Clearly, this produces a sharper image than the above 2 methods. See the white patch on the left side of the apple. This method balances processing time and output quality fairly well.

https://theailearner.com/2018/11/15/image-interpolation-using-opencv-python/

General Interpolation: Summary

- For NN interpolation, the output pixel is assigned the value of the pixel that the point falls within. No other pixels are considered.
- For bilinear interpolation, the output pixel value is a weighted average of pixels in the nearest 2-by-2 neighborhood.
- For bicubic interpolation, the output pixel value is a weighted average of pixels in the nearest 4-by-4 neighborhood.
- Bilinear method takes longer than nearest neighbor interpolation, and the bicubic method takes longer than bilinear.
- The greater the number of pixels considered, the more accurate the computation is, so there is a trade-off between processing time and quality.

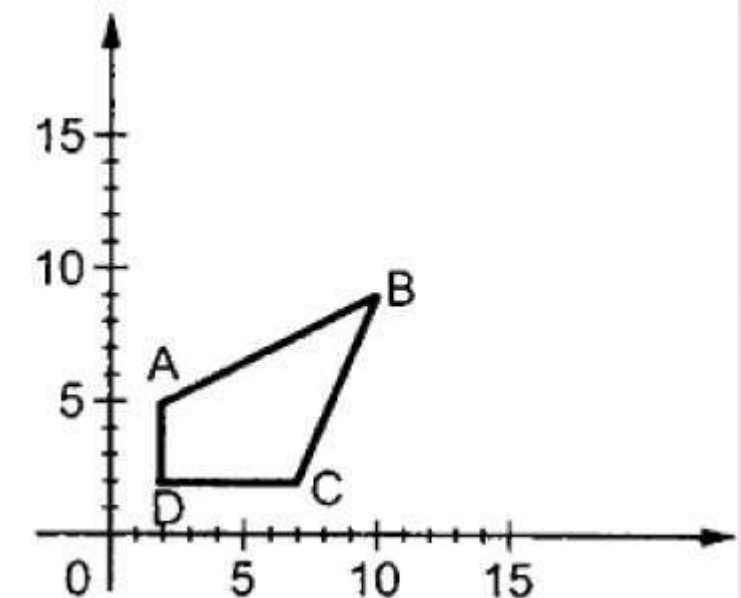
General Interpolation: Summary

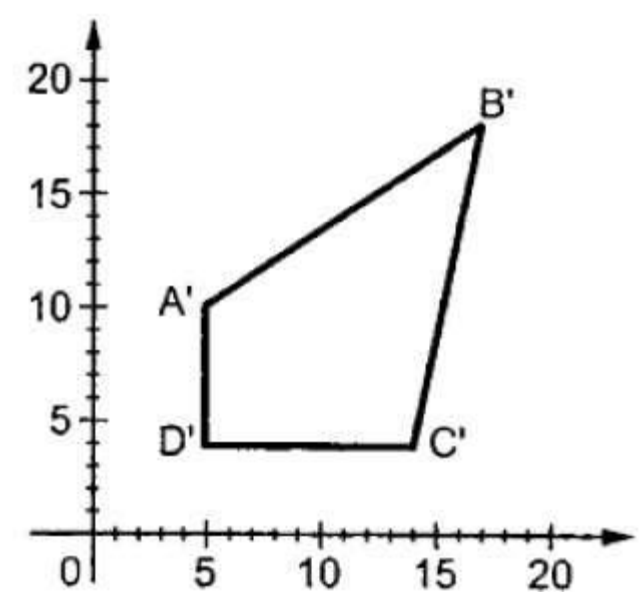


https://en.wikipedia.org/wiki/File:Comparison_of_1D_and_2D_interpolation.svg

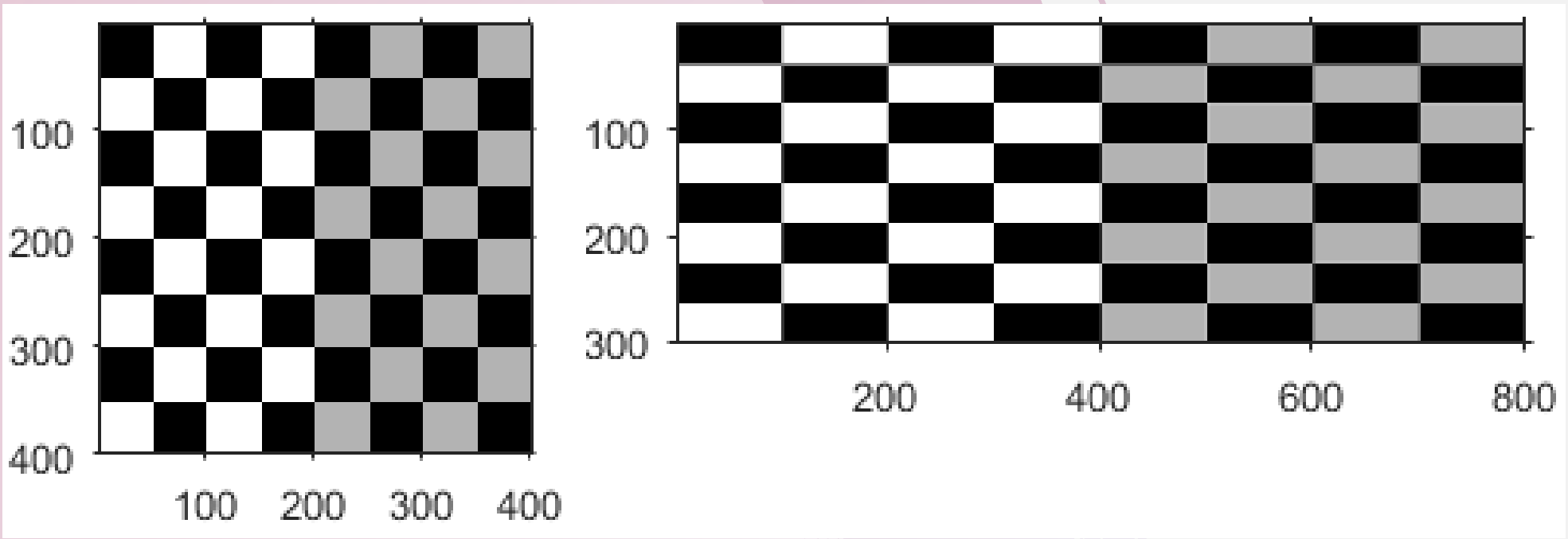
Scaling Operation

- To shrink or zoom the size of an image (or part of an image).
- To change the visual appearance of an image;
- To alter the quantity of information
- To use as a low-level pre-processor in multi-stage image processing chain which operates on features of a particular scale.
- Scaling is a special case of *affine transformation.*



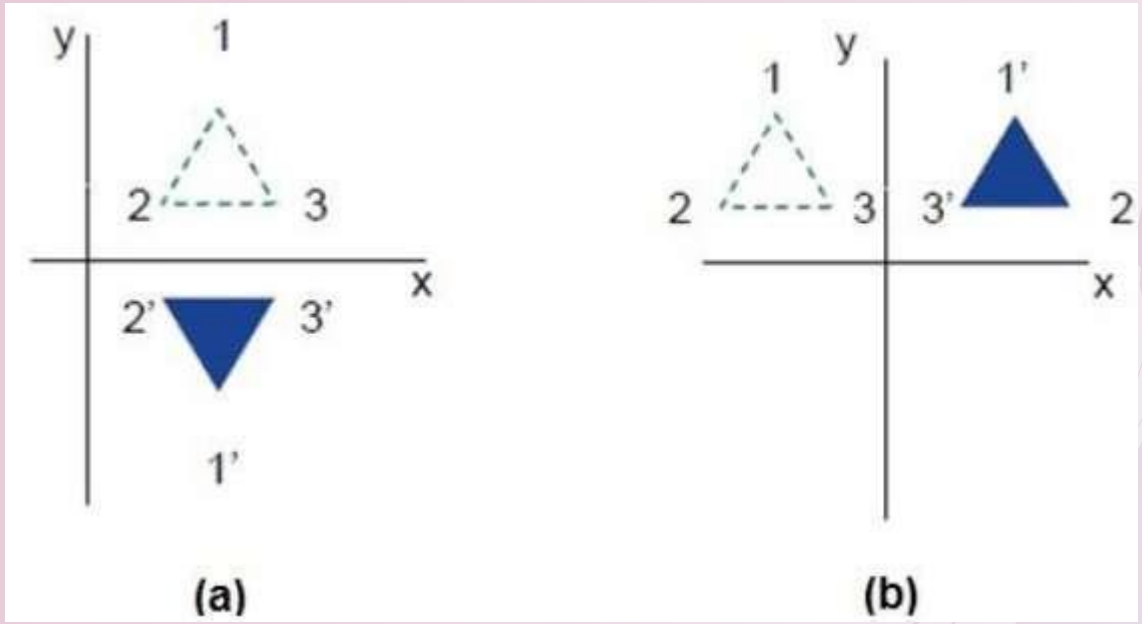


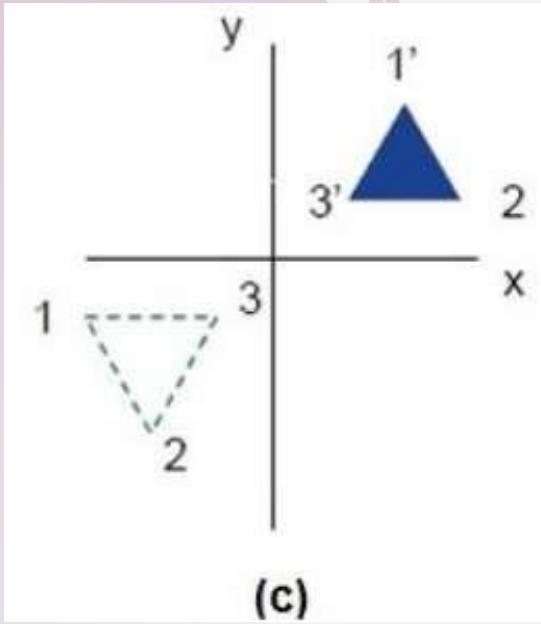
Scaling Operation

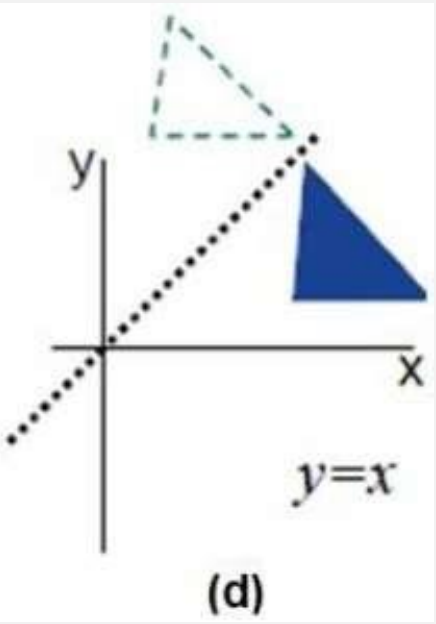


Reflection / Flip / mirror

- Reflection is the mirror image of original object. In other words, we can say that it is a rotation operation with 180°. In reflection transformation, the size of the object does not change.

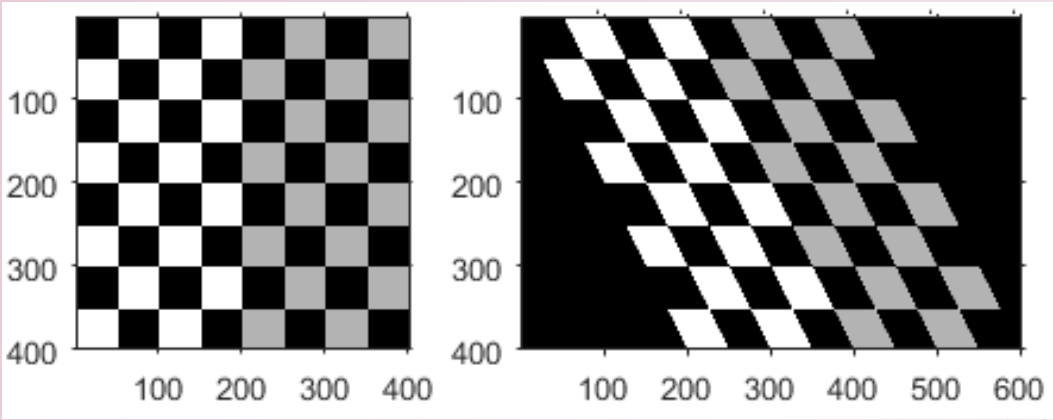






Shear Operation

- Shear tool is used to shift one part of an image, a layer, a selection or a path to a direction and the other part to the opposite direction.
- For instance, a horizontal shearing will shift the upper part to the right and the lower part to the left. A rectangle becomes a diamond.





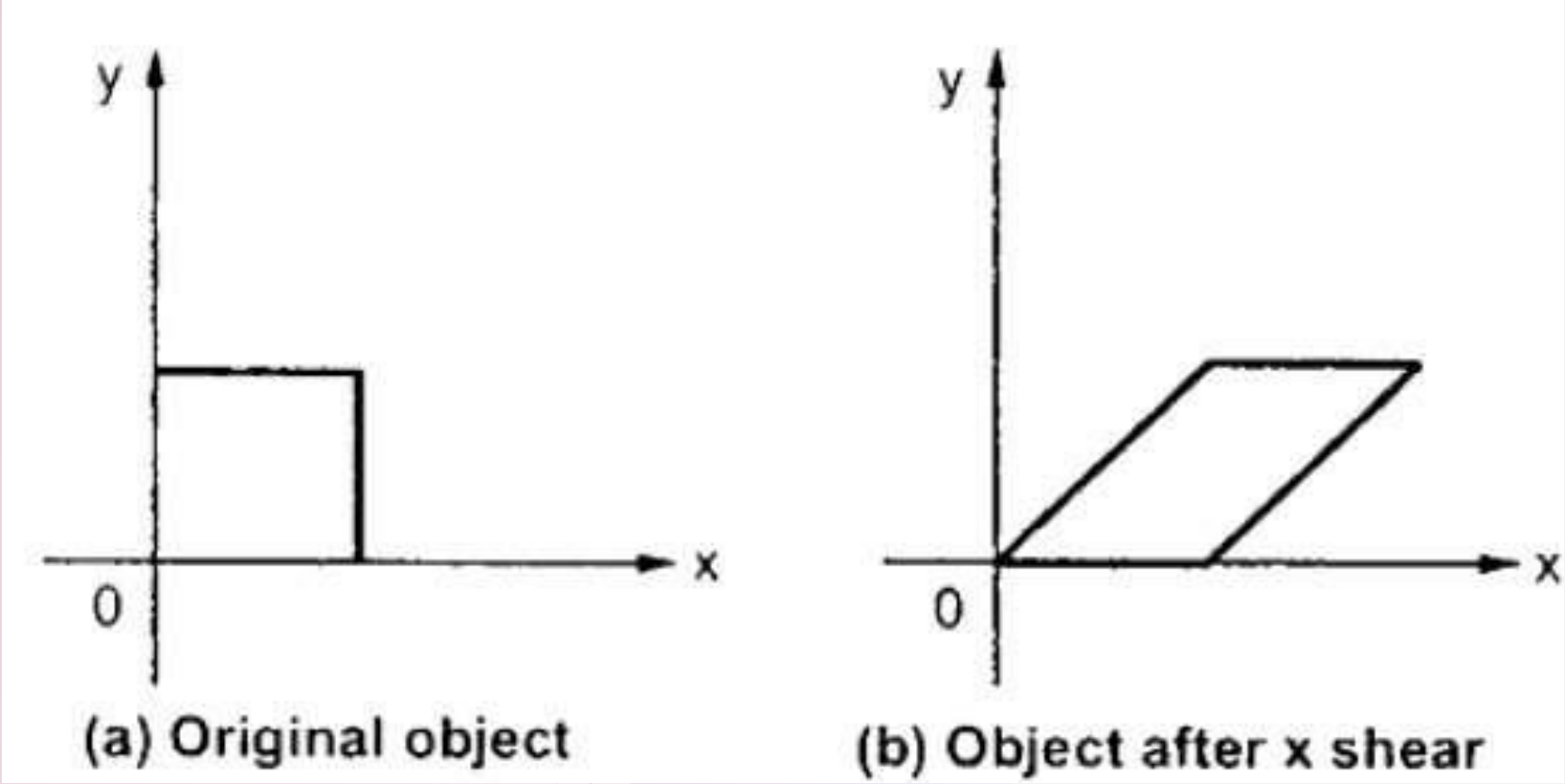
https://docs.gimp.org/2.10/en/gimp-tool-shear.html

Shear Operation

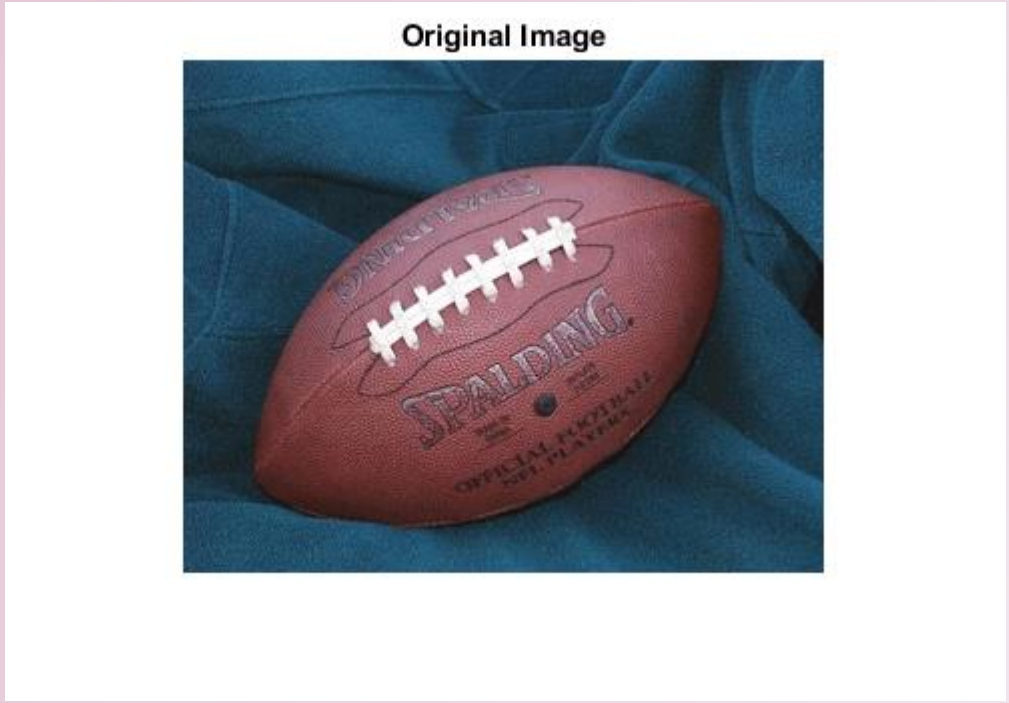
In two dimensions, a simple shear transformation that maps a pair of input coordinates [u v] to a pair of output coordinates [x y] has the form

$$x = u + a * v$$

$$y = v$$



Shear Operation

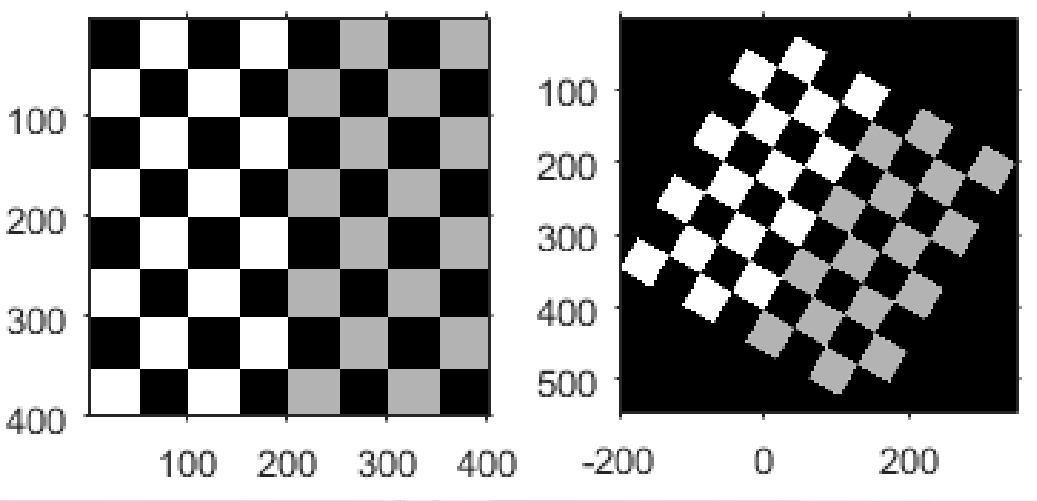




https://www.mathworks.com/help/images/padding-and-shearing-an-image-simultaneously.html

Rotation Operation

- A geometric transform which maps the position of a picture element in an input image onto a position in an output image by rotating it through an angle about an origin.
- Commonly used to improve the visual appearance of an image.
- Can also be useful as a pre-processor in applications where directional operators are involved.
- Rotation is a special case of affine transformation.

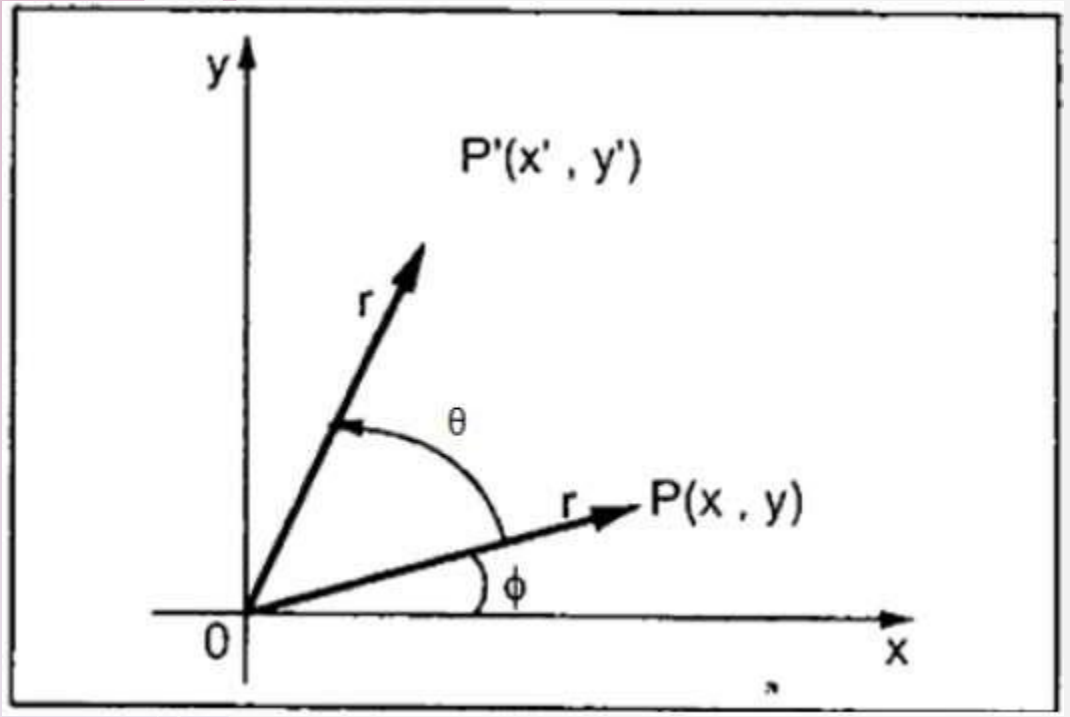


Rotation Operation (cont)

- Mapping of a point (x,y) to another (x',y') through a counter-clockwise rotation of θ

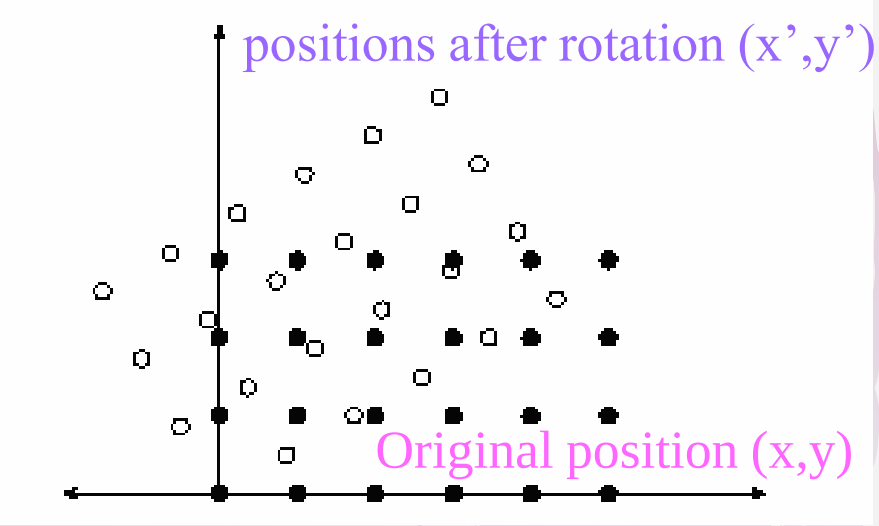
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix} \begin{bmatrix} x' \\ y' \end{bmatrix}$$



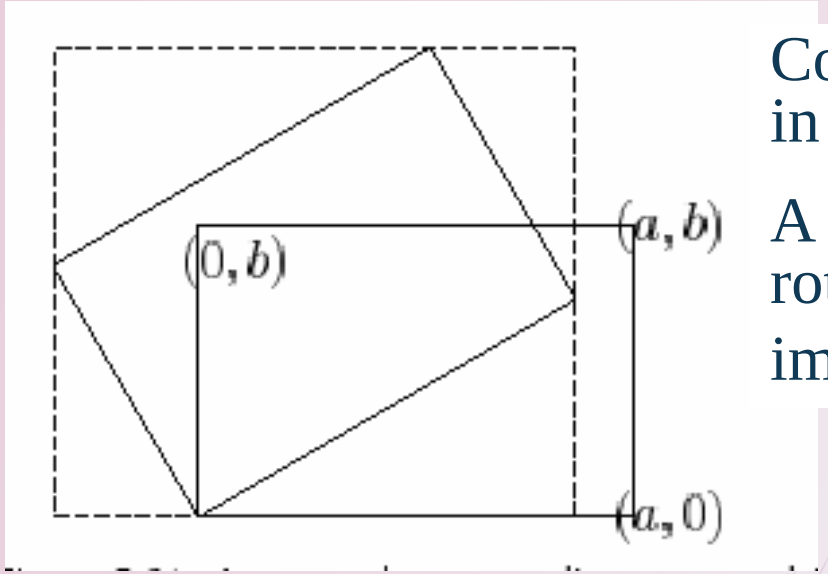
Rotation Operation: Problems?

- In image space, when rotating a collection of points, what could go wrong?



Rotation Operation: Remedies

- Problem1: part of rotated image might fall out of valid image range.
- Problem2: how to obtain the intensity values in the rotated image?



A rectangle surrounding a rotated image

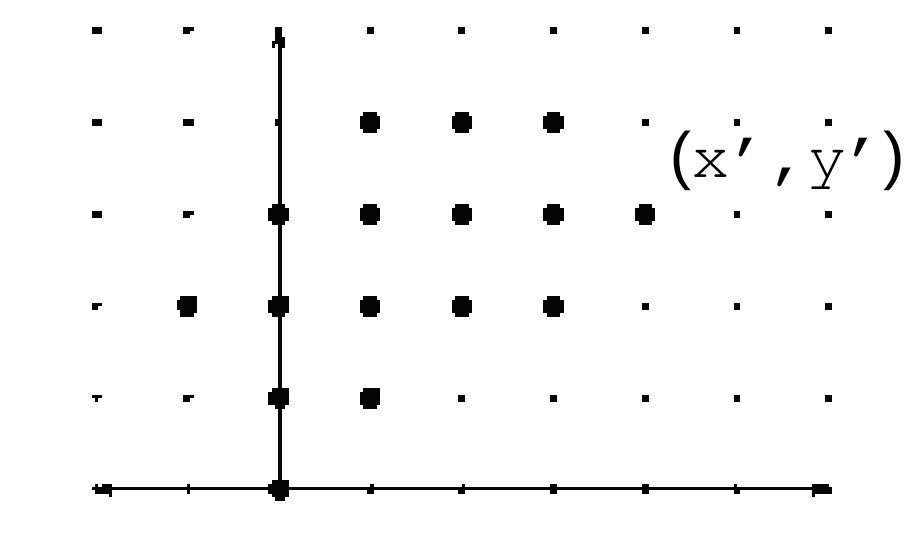
Consider all integer-valued points (x',y') in the dashed rectangle.

A point will be in the image if, when rotated back, it lies within the original image limits.

$$0 \le x'\cos\theta + y'\sin\theta \le a$$

$$0 \le -x'\sin\theta + y'\cos\theta \le b$$

Rotation Operation: Remedies (con'd)

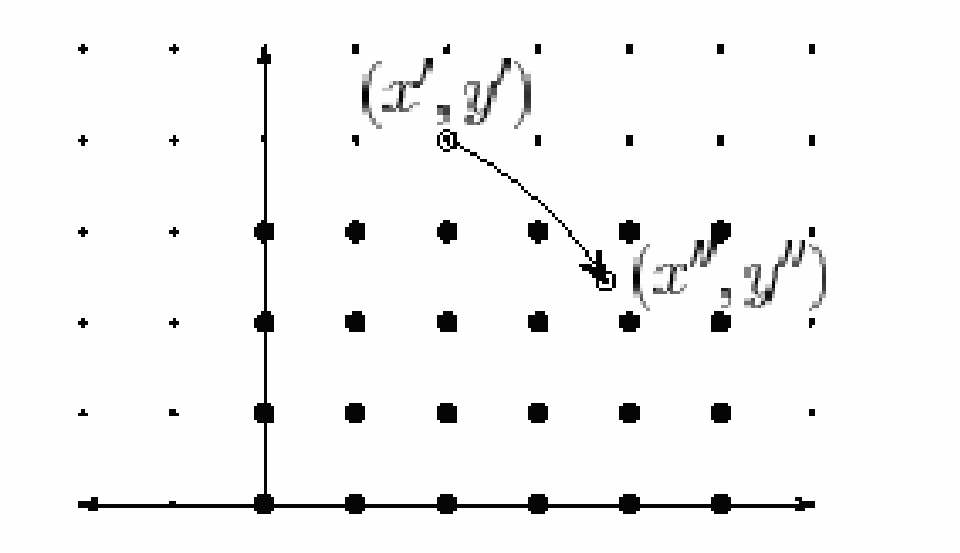


(x′,y′) in rotated image

(x",y") is the rotated (x′,y′) back into the original image

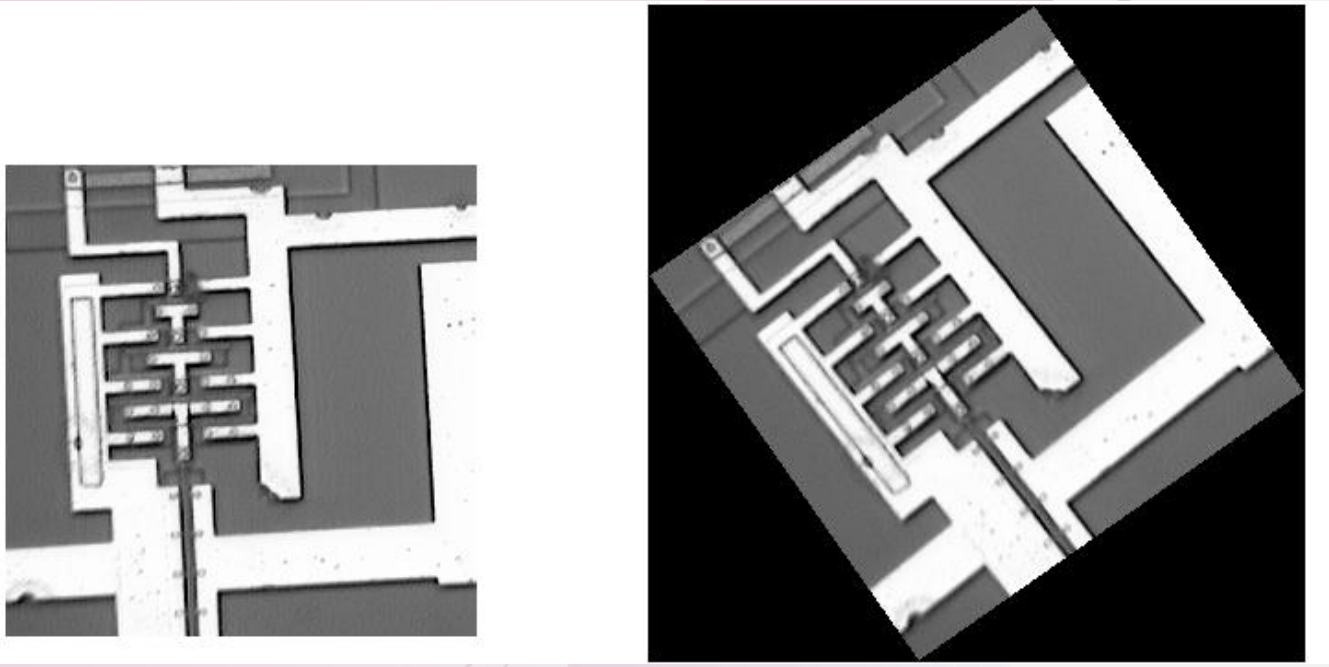
• The grey value at (x",y") can be found by interpolation.

• This value is the grey value for the pixel at (x′,y′) in the rotated image.



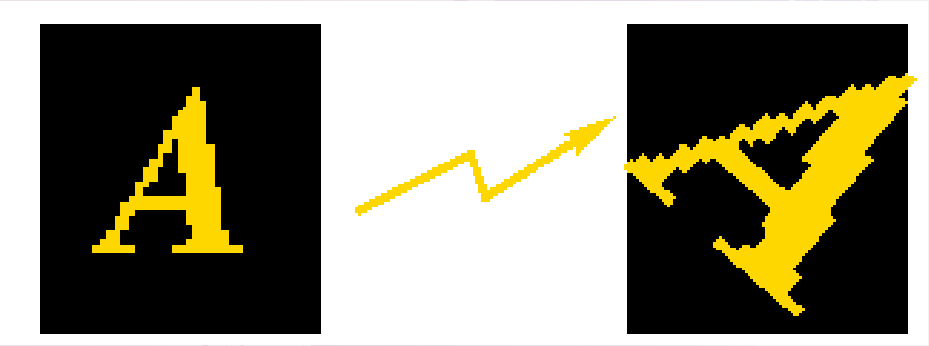
Rotation Operation: Example

```
I = imread('ic.tif');
J = imrotate(I,35,'bilinear');
imshow(I)
figure, imshow(J)
```



Affine Transformation

- An affine transformation maps variables (*e.g.* pixel intensity values located at position in an input image) into new variables (*e.g.* in an output image) by applying a linear combination of translation, rotation, scaling operations.
- **Significance:** In some imaging systems, images are subject to geometric distortions. Applying an affine transformation to a uniformly distorted image can correct for a range of perspective distortions.



Affine Transformation (con'd)

- An affine transformation is equivalent to the composed effects of translation, rotation and scaling, and shearing.
- The general affine transformation is commonly expressed as below:

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = A \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + B$$

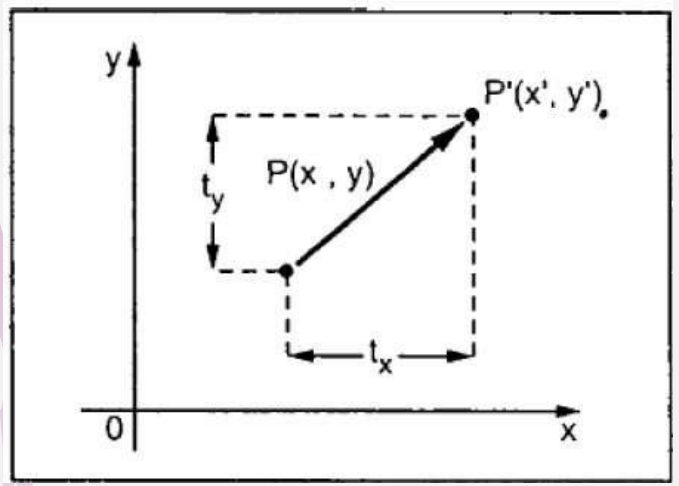
0th order coefficients

1st order coefficients

Affine Transformation (con'd)

- By defining only the *B* matrix, this transformation can carry out pure **translation**:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, B = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$



- Pure **rotation** uses the *A* matrix and is defined as (for positive angles being clockwise rotations):

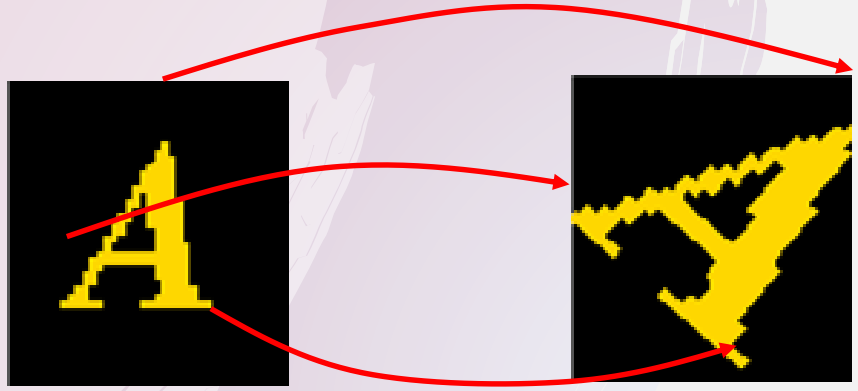
$$A = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}, B = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Affine Transformation (con'd)

- Pure **scaling** is defined as

$$A = \begin{bmatrix} a_{11} & 0 \\ 0 & a_{22} \end{bmatrix}, B = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

- Since the general affine transformation is defined by 6 constants, it is possible to define this transformation by specifying 3 corresponding point pairs (more in next class).



Example

```
[ ]  def rotate_image(image, degree, flags=cv2.INTER_LINEAR):
       h, w = image.shape[:2]
       (cX, cY) = (w // 2, h // 2)
       M = cv2.getRotationMatrix2D((cX, cY), degree, 1.0)
       rotated = cv2.warpAffine(image, M, (w, h), flags=flags)
       return rotated
```

```
[ ]  def show_rotate(image, rotate, d):
       fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, sharex=True, sharey=True, figsize=(15,15))
       ax1.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
       ax1.title.set_text("original")

       nearest = rotate(image, d, flags = cv2.INTER_NEAREST)
       ax2.imshow(cv2.cvtColor(nearest, cv2.COLOR_BGR2RGB))
       ax2.title.set_text("inter nearest")

       linear = rotate(image, d, flags = cv2.INTER_LINEAR)
       ax3.imshow(cv2.cvtColor(linear, cv2.COLOR_BGR2RGB))
       ax3.title.set_text("inter linear")

       cubic = rotate(image, d, flags = cv2.INTER_CUBIC)
       ax4.imshow(cv2.cvtColor(cubic, cv2.COLOR_BGR2RGB))
       ax4.title.set_text("inter cubic")
```

Example


original
0
100
200
300
400
500
CASIO
TELE MEMO 20
DATA BANK
W-R50
THU 5-28
2:5039
ADJUST
REVERSE/LIGHT
MODE/SELECT
FORWARD/24HR
SEARCH

inter nearest
CASIO
TELE MEMO 20
DATA BANK
W-R50
THU 5-28
2:5039
ADJUST
REVERSE/LIGHT
MODE/SELECT
FORWARD/24HR
SEARCH


inter linear
0
100
200
300
400
500
CASIO
TELE MEMO 20
DATA BANK
W-R50
THU 5-28
2:5039
ADJUST
REVERSE/LIGHT
MODE/SELECT
FORWARD/24HR
SEARCH


inter cubic
0
100
200
300
400
500
CASIO
TELE MEMO 20
DATA BANK
W-R50
THU 5-28
2:5039
ADJUST
REVERSE/LIGHT
MODE/SELECT
FORWARD/24HR
SEARCH

Example

```
[ ]  def shear_image(image, s = 0.5, flags=cv2.INTER_LINEAR):
       h, w = image.shape[:2]
       T = np.float32([[1, s, 0], [0, 1  , 0]])
       sheared = cv2.warpAffine(image, T, (w, h), flags=flags)
       return sheared
```

Summary

- Interpolation of intensity values on non-grid points:
  - Nearest Neibhgor (NN)
  - Bilinear
  - Bicubic
- Image transformation
  - Computation of intensity values of the transformed image
  - Discussed some instances of affine transformation
    - Translation
    - Rotation
    - Scaling



THỰC HÀNH

Thực hành

Phân tích ảnh hưởng của ít nhất 4 phép biến đổi ảnh khi thực hiện tăng cường dữ liệu cho bài toán:

- Bài toán 1: Phân loại ảnh (Classification)
  - MobileNetv2
  - Dataset Caltech101
- Bài toán 2: Dò tìm đối tượng trong ảnh (Object Detection)
  - Chọn Yolo hoặc Faster RCNN
  - Dataset Pascal VOC2007

Thực hành

Phương pháp	Độ đo đánh giá
Không tăng cường dữ liệu	X0
Tăng cường dữ liệu với phép biến đổi T1	X1
Tăng cường dữ liệu với phép biến đổi T2	X2
Tăng cường dữ liệu với phép biến đổi T3	X3
Tăng cường dữ liệu với phép biến đổi T4	X4
Tăng cường dữ liệu với phép biến đổi T1+T2+T3+T4	X5

Thực hành

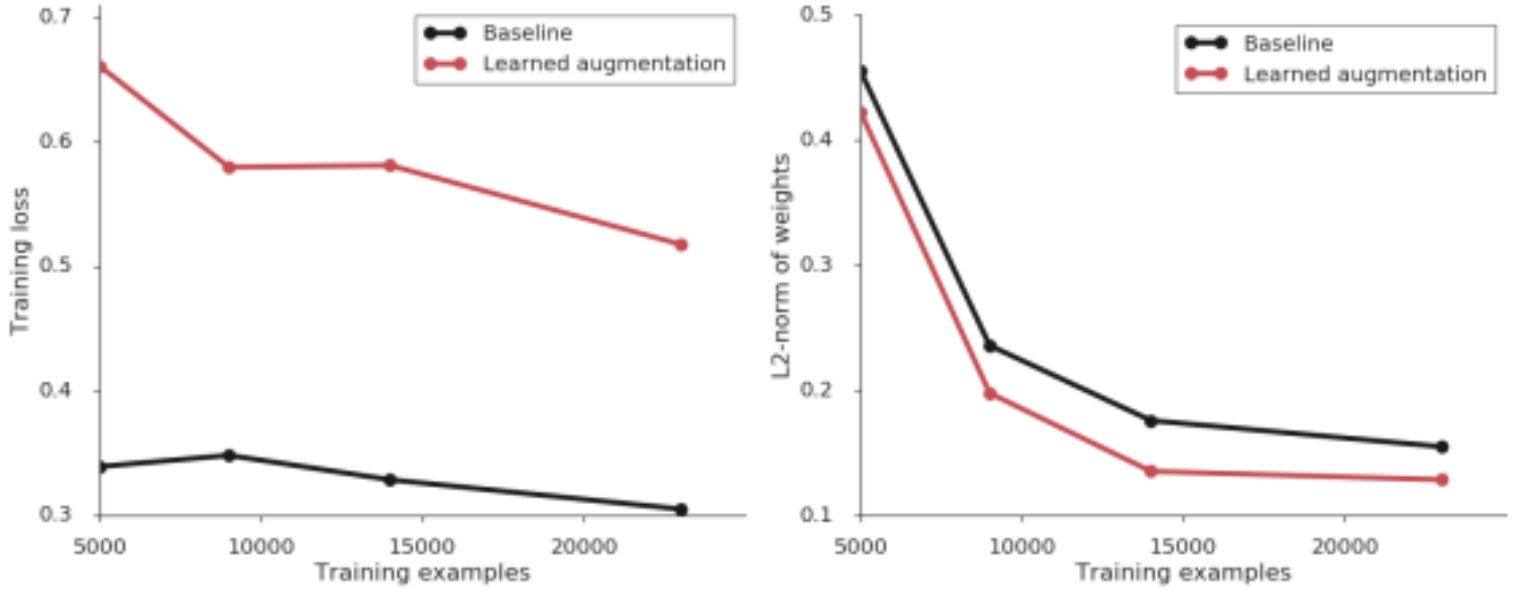


Fig. 4. Two plots showing data augmentation regularizes detection models. On the left is training loss vs. number of training examples for baseline model (black) and with the data augmentation policy (red). One the right is L_2 norm of the weights of the baseline (black) and our (red) models at the end of training. Note that the L_2 norm of the weights decrease with increasing training set size. The data augmentation policy further decreases the norm of the weights.

https://www.ecva.net/papers/eccv_2020/papers_ECCV/papers/123720562.pdf

Tài liệu tham khảo

- https://fairyonice.github.io/Learn-about-ImageDataGenerator.html
- https://www.tensorflow.org/tutorials/images/data_augmentation
- https://neptune.ai/blog/data-augmentation-in-python
- https://www.pyimagesearch.com/2019/07/08/keras-imagedatagenerator-and-data-augmentation/
- https://www.slideserve.com/alida/digital-image-processing-lecture-7-geometric-transformation
- https://homepages.inf.ed.ac.uk/rbf/HIPR2/rotate.htm
- https://boxhoidap.com/huong-dan-bilinear-interpolation-image-processing-python-python-xu-ly-hinh-anh-noi-suy-song-tuyen